

Project Report for FlickPond: A Reusable Cloud Media Processing Platform

Practice Module for Graduate Certificate in Architecting Scalable Systems

Team 17

Members:

Guruprasath Gopal

Ibrahim Mammadov

Nguyễn Kim Long

SWE5001 Architecting Scalable Systems · SE34FT · September–November 2026

Repository: github.com/Collaboration95/dc2 · Report version 1.0, 21 October 2026

CONTENTS

1. INTRODUCTION 4
   1.1 Background 4
   1.2 Business Needs 5
   1.3 Stakeholders 6
   1.4 Project Scope 6
2. PROJECT CONDUCT 9
   2.1 Project Plan 9
   2.2 Project Status 10
   2.3 Project Metrics 11
3. SOLUTION OVERVIEW 13
   3.1 Logical Architecture & Design 14
   3.2 Physical Architecture & Design 25
   3.3 Other Architectural Decisions 36
   3.4 Architectural Limitation 37
4. QUALITY ATTRIBUTES 39
   4.1 Performance 41
   4.2 Availability 41
   4.3 Security 42
   4.4 Extensibility and Maintainability 44
5. DEVOPS AND DEVELOPMENT LIFECYCLE 45
   5.1 Source Control Strategy 45
   5.2 Continuous Integration 46
   5.3 Continuous Delivery 48
6. OTHER THINGS TO BE HIGHLIGHTED 50
   6.1 What Team 17 Contributed beyond the Inherited Baseline 50
   6.2 Lessons Learned 51
   6.3 Responsibilities for Use Cases and Design Problems 51
   6.4 Evidence and How to Reproduce It 52
   6.5 AI Tool Declaration 52

7. APPENDICES 52
7.1 Use-Case Specifications and Traceability 52
7.2 Analysis and Design Models 55
7.3 Design Problems 61
7.4 Glossary 65

1. Introduction

1.1 Background

FlickPond began as a Project 1 video application. Users register and upload their own videos. FFmpeg workers turn each upload into an MP4 and adaptive HLS streams (360p, 480p and 720p), and the owner then plays the result through an owner-checked link. Project 1 was built by a five-member FlickPond team. It ran FastAPI, PostgreSQL, Redis with RQ, MinIO object storage, nginx and a browser frontend on one cloud virtual machine. It worked, but as **one application**: a single API handled identity, uploads, jobs and playback; the API, the workers and the recovery task shared one job model and one database; and the jobs table had a foreign key to the users table. Another product could not reuse the video workflow without importing FlickPond's code or reading its database.

Team 17 was formed for this practice module from members of two Project 1 teams. Prof Boon Kui Heng approved the three-member team on 22 September 2026. Ibrahim Mammadov was a member of the FlickPond Project 1 team (track A: worker, queue and media-type checks). Guruprasath Gopal and Nguyễn Kim Long come from a different Project 1 team. The team chose to carry the frozen FlickPond code forward as its baseline, because it already contained a real, CPU-heavy asynchronous workload. The submitted proposal names repository `Flickpond/Media_player`, commit `890c5ec`. The working plan used the team's local checkout of the same frozen code, commit `8854537` (decision D-01, 9 October). No Project 1 feature work was continued, and inherited capabilities are not claimed as Project 2 work.

The proposal (4 October 2026) set the goal: turn FlickPond into a **reusable media processing platform**. Applications would use one documented API for upload, processing, status and playback, with processing running as an independently owned and scaled service. The lecturer accepted the proposal and asked the team to confirm the microservice choice through detailed domain analysis (feedback F-01). Section 3.1.4 records that analysis.

Current status (21 October 2026). The platform is built, deployed on AWS and verified.

- Two business services (**Media Management** and **Processing**) each own their database, image, IAM roles, CI lane and release unit. They communicate only through versioned messages on Amazon SQS. Neither calls the other synchronously.
- Two client applications use only the public API: the FlickPond **web app** and a new **Publisher CLI**.
- The persistent AWS environment `dc2-live` (ap-southeast-1) has run since 18 October. It scales to zero when idle, and the CD pipeline updates it in place after every green CI run on `Main`.
- All seven use cases in the proposal are implemented, with automated tests and recorded cloud evidence. Measured misses against the proposal targets are reported as misses (section 4).
- The team presents on **26 October 2026, 14:35–15:15 SGT**. The final report is due on **16 November 2026**. The environment is torn down once, after that date.

The table summarises what Team 17 inherited and what it added.

Area	Inherited from FlickPond (Project 1)	Added by Team 17 (Project 2)
Service structure	One FastAPI application; workers and recovery task share its job code	Two bounded contexts, Media Management and Processing, found through DDD and connected only by asynchronous, versioned contracts
Data ownership	One database; jobs reference users	`media_db` and `processing_db` with separate migration histories and least-privilege roles; no cross-database keys
Reuse	One browser application	A second, independent client (Publisher CLI) built against the published API only
Runtime	One VM with nginx, Redis/RQ and MinIO	Serverless AWS: CloudFront, API Gateway, Lambda container images, SQS, Aurora Serverless v2, S3 and finite ECS Fargate encoders that scale 0 → 3 → 0
Security	Login, owner checks, TLS, signed output links, CI scans	Media-owned Gateway authorizer, trusted actor context, per-handler IAM roles, queue resource policies, bounded direct-to-S3 upload grants, pinned source versions, cloud negative tests
Delivery	Tests and scans in CI; manual deployment	Path-filtered per-service CI lanes, image scans and DAST; GitHub Actions CD over OIDC with per-owner release, smoke test and automatic restore
Evidence	Load tests that mostly read one completed video	Scaling, latency, recovery, release, security and reuse experiments with retained receipts

FFmpeg encoding, HLS output, worker replicas and the basic CI workflow are inherited capabilities. Team 17 reshaped and reused them, but does not count them as its own contribution.

1.2 Business Needs

Small content teams keep needing the same video capabilities. Training providers, course teams, small studios and community publishers are examples. They need:

- secure upload of videos they own;
- conversion into formats that play on any device;
- visibility of processing progress and failures;
- playback restricted to the owner.

Today each application rebuilds these capabilities or copies another team's code. That duplicates effort and produces inconsistent security and operations. There is also a capacity problem: transcoding is CPU-heavy and bursty. When it shares a deployment unit with the user-facing API, a backlog of uploads competes with logins and status checks.

The business drivers for the platform follow from this:

1. **Reuse.** A new application gets authenticated upload, processing, status and owner-checked playback by calling one published API, with no copied code and no database access.
2. **Independent capacity.** Processing is its own service. Encoder capacity follows the backlog without scaling or redeploying the user-facing API.
3. **Repeatable, secure releases.** Every change passes automated tests and security gates. Each service is then released on its own from an immutable image and verified by an automated upload-to-playback smoke test.
4. **Low idle cost.** A small team's platform is idle most of the time, so it should cost almost nothing when idle. The design target became scale-to-zero: about USD 3–5 a month at idle (AD-27).

In platform terms (section 3.1.4), the **seed** is an owned video asset and its processing job. **Producers** are content owners submitting videos through the web app or the Publisher CLI. **Consumers** are owners playing their processed output. **Integrators** build new clients against the OpenAPI contract. **Operators** release and scale the platform.

1.3 Stakeholders

The project has no external sponsor; it is team-proposed. Business stakeholders are represented by four planning personas defined in week 1 (`project-roadmap/business-scope.json`). They are fictional design assumptions, not interviewed customers.

Stakeholder	Type	Interest and need	Use cases
Content owners, e.g. course coordinator “Maya” (P-01)	Business: primary producer	Submit a folder of recordings once, see each result, retry failures and check playback	UC-1 to UC-5
Independent trainers, e.g. “Arun” (P-02)	Business: secondary producer and viewer	Upload one video in the browser, follow its status, watch the result; nobody else can see it	UC-1 to UC-4
Application developers, e.g. “Jules” (P-03)	Business/IT: integrator	Use the published API without copying service code or holding database credentials	UC-1, UC-3 to UC-5
Platform operators, e.g. “Sam” (P-04)	IT: operations	Change processing capacity and release Processing without touching Media; see failures and recover	UC-6, UC-7
Course lecturer and assessors (Prof Boon Kui Heng, NUS-ISS)	Academic sponsor and reviewer	Approved the three-member team and the proposal; assesses the architecture, evidence and delivery	All
Team 17 members	Delivery team	Guruprasath Gopal (cloud and delivery), Ibrahim Mammadov (backend services), Nguyễn Kim Long (domain, contracts and verification)	All
FlickPond Project 1 team	Upstream code owner	Authors of the inherited baseline; credited for inherited capabilities	Baseline
Amazon Web Services	IT: cloud provider	Hosts the managed services in `ap-southeast-1`	Runtime
GitHub	IT: tooling provider	Source control, pull requests, Actions CI/CD, Pages for the study guide	Delivery

1.4 Project Scope

The scope is the proposal’s seven use cases (UC-1 to UC-7), delivered as two services, two clients and one AWS environment. There are three levels of scope:

- **Architected.** The two-context platform, its contracts, the AWS topology and the operating model.
- **Implemented.** Everything architected, plus three opt-in experiments kept off by default (AD-22): an unattended encoder launcher (later adopted as the V2-live default), short encodes on Lambda, and one database cluster per service.
- **Experimentally verified.** The results in section 4. Each result is labelled with the environment that produced it: local, AWS V1 per-run stack, or the persistent V2-live environment.

The project ran in three phases.

Phase	Dates	Delivered
0. Baseline and design	22 Sep – 9 Oct	Team approval, proposal (4 Oct), lecturer feedback, frozen baseline, personas and use cases, DDD model, contracts, build preparation
1. V1: two services on AWS, per-run stacks	10 – 18 Oct	Processing extracted (12 Oct), Media and Publisher adapted (13 Oct), AWS template and pipeline (15 Oct), cloud proof runs `r1017b`, `r1018b`, `r1019a`; tag `project2-v1` (18 Oct)
2. V2-live: one persistent environment	18 Oct – 16 Nov	`dc2-live` deployed (18 Oct), automatic in-place CD, unattended 0 → 3 → 0 encoders, security negatives, rollback proof, freeze (20 Oct), engineering hardening, presentation (26 Oct), final report (16 Nov)

The detailed scope sources are the proposal (§3), the use-case catalogue (`docs/architecture/use-case-design.md`) and the proposal-to-as-built deviation table (`docs/architecture/decisions.md` §4). Section 3 explains each deviation.

1.4.1 Functionality in scope

ID	Use case	Clients	Status
UC-1	Register, log in and upload an owned video (100 MiB maximum)	Web app, CLI	Implemented. Direct-to-S3 upload with a 15-minute owner-scoped grant; bytes verified and the S3 version pinned before the asset exists
UC-2	Process the video into MP4 and HLS (360p/480p/720p, never above the source height); HLS is best effort	Platform	Implemented on finite Fargate encoders
UC-3	Track job status and play the processed result	Web app, CLI	Implemented. Owner-only status from Media's projection; MP4 and HLS through short-lived signed URLs
UC-4	Retry a failed job using the stored source	Web app, CLI	Implemented. A new fenced attempt over the same pinned source; no re-upload
UC-5	Bulk-publish a folder of videos and report each result	CLI	Implemented in the Publisher CLI
UC-6	Scale processing capacity without touching Media Management	Operator / platform	Implemented. 1–3 operator batches (V1); automatic 0 → 3 → 0 from the SQL backlog (V2-live)
UC-7	Release a new Processing version on its own, with smoke test and rollback	Operator / pipeline	Implemented. Per-owner release by image digest and Lambda alias, with automatic restore on a failed smoke

Figure 1 shows the actors and the use cases, in scope and out of scope.

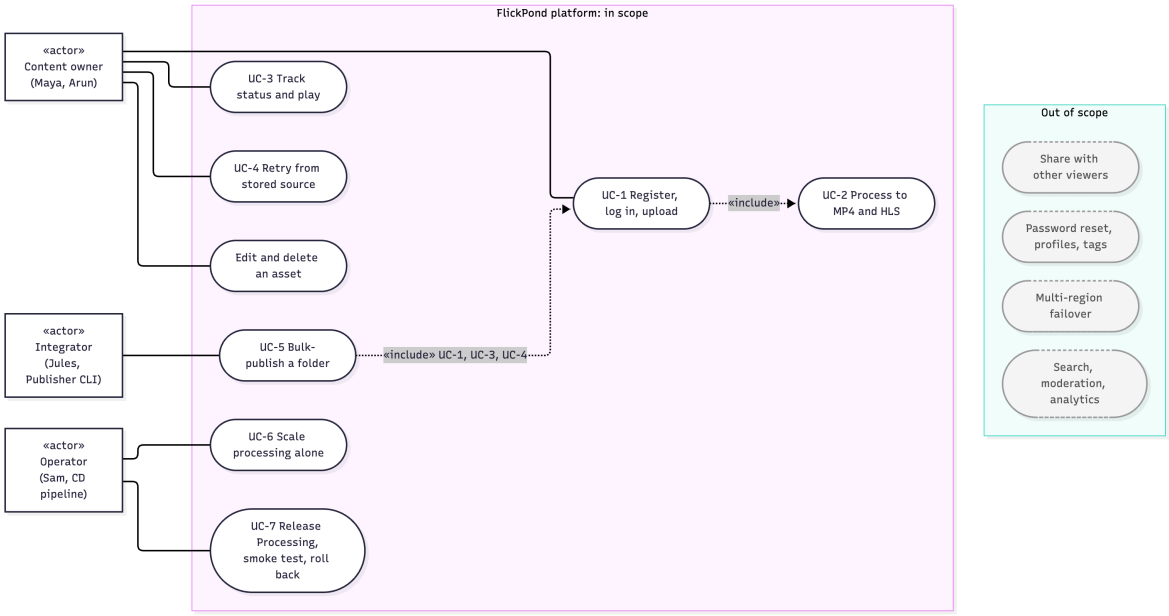


Figure 1. Use-case diagram: actors, the seven in-scope use cases with the added edit and delete capability, and the main out-of-scope cases.

The as-built platform also has three capabilities the proposal did not list. Each is small, and each reuses the same aggregates:

- Owner **edit** operations (clip, crop, scale, convert) that create a derived asset.
- **Delete**, recorded as a tombstone in both contexts and followed by source cleanup.
- Operator-only **admin** routes.

1.4.2 Functionality out of scope

These requirements are excluded, as recorded in the proposal (§3.5) and the briefing interpretation:

- The rest of the Project 1 feature list: profiles, password reset, titles and tags, sharing modes and external videos.
- Viewer identities other than the owner: public sharing, student or LMS access, shared team ownership, tenants and billing. The contract accepts a `viewer` role that nothing issues.
- Search, public discovery, moderation workflows and product analytics.
- Crop or clip editor UI, thumbnails, 4K output, and resumable uploads above 100 MiB.
- Kubernetes, general-purpose autoscaling, and multi-AZ database or multi-region failover.
- Historical data migration, malware scanning and full key rotation.
- A third client, a developer portal, live streaming, and a separate staging environment.

One earlier planning exclusion was superseded. The proposal excluded autoscaling, but V2-live **does** start finite encoders automatically from the SQL backlog, bounded by a hard cap of three (AD-27). This is bounded task launching, not a general autoscaling service.

2. Project Conduct

2.1 Project Plan

The briefing expects about 10 person-days (pd) per participant. The submitted proposal estimated **31 pd** for three members across ten work packages (WBS). Each package had a lead member (proposal §4.1).

#	Work package	Lead	Estimate (pd)
1	Freeze baseline, reproducible local setup, fix the inherited scan finding	Guruprasath	2
2	Domain analysis, service contracts and OpenAPI documents	Long (team review)	3
3	Extract Processing: service, own database and migrations, workers and recovery task	Ibrahim	6
4	Adapt Media Management: asset records, Processing client, web app routes and retry	Ibrahim	3
5	Boundary security: identity, owner checks, database roles, presigned URLs, negative tests	Guruprasath	3
6	Publisher CLI and contract tests	Long	2
7	AWS environment, per-service deployment, pipeline updates, smoke test	Guruprasath	4
8	Experiments: scaling, load, recovery and playback; raw results	Long	3
9	Integration fixes and repeat checks	All	2
10	Progress report, presentation and final report	All	3
	Total		**31**

The proposal's schedule had six periods:

- 5–7 October: baseline and domain model.
- 8–14 October: extraction and the web app end to end.
- 15–20 October: security, AWS, pipeline and experiments.
- 21–25 October: fixes and rehearsal.
- 26 October: the presentation.
- 27 October – 16 November: the final report.

The plan recorded four main risks:

- Tangled shared code.
- AWS account or cost issues.
- Little scaling gain.
- A three-member team.

Each had a mitigation: extract first; set up AWS in week 1 with cost alerts; cap CPU per worker and report honestly; narrow the scope.

Way of working.

- **Backlog.** Work was planned as GitHub issues with outcome, acceptance and verification sections. The issues sat on a GitHub Project board (Project 14) with Workflow, Wave, Priority, Risk and Area fields.

- **Delivery waves, not timeboxed sprints.** Wave 0–4 locally, then the AWS V1 bundles, then close-out.
- **Integration.** Pull requests went into integration branches (section 5.1), with CI on every change.
- **Decisions.** Recorded as ADRs and project decisions (F-01, D-01 to D-07).
- **Reporting.** One progress report was due on 16 October (decision D-06).

There were no formal sprint reviews or retrospectives. Their place was taken by:

- the 9 October planning review;
- independent plan reviews (`docs/execution/reviews/`);
- the 18 October audit and board reconciliation (#253, #258);
- the 20 October freeze, with its acceptance checklist.

2.2 Project Status

As of 21 October 2026, the project is complete for its committed scope, and is in the presentation and report phase.

Area	Status
Use cases UC-1 to UC-7	All implemented and tested; cloud evidence for each (section 4)
Architecture	Two bounded contexts; asynchronous contracts; persistent AWS environment (AD-27)
Live environment	`dc2-live` up since 18 October; scales to zero; torn down once, after 16 November
Release identity	Team-accepted demo freeze at `daee44b` (20 October). Later engineering release `629119c` (20 October), verified end to end with an upload-to-playback smoke and a read-back of all images and aliases. The 21 October scan-policy fix is recorded below.
CI	Latest `Main` CI run 39531399249: 25 jobs passed, 3 intentional skips, 7 min 9 s
CD	Run 39532075705 (21 October) **failed** at the release scan of the retained Media rollback image. Fixed by AD-28 and redelivered (section 5.3).
Evidence	Results against every proposal target, including three measured misses (section 4)
Submission	Presentation 26 October 14:35–15:15 SGT; code submission 31 October (internal date); final report 16 November

Outstanding issues.

Item	Disposition
Three measured misses: two-worker scaling 1.449× (target 1.6×); Processing release 10.1–10.3 min (target 10 min); queue wait about 37 s (target 20 s)	Reported against the unchanged targets. Improvement options are in AISS-03 and AISS-04 (section 3.4).
Evidence not established on AWS: lost-queue-entry timing, duplicate-output check after a worker kill, a restore drill, actual spend	AISS-14. The final October bill is read before 16 November.
Optional V2 experiments #68 and #80	#68 (unattended launcher) was adopted into V2-live. #80 (separate V2 branches) is not needed, because the experiments live behind template flags (AD-22).
Presentation deck and rehearsal (#192), recordings (#254)	Prepared from this report and the frozen demo release before 26 October
Presentation feedback	Incorporated into the final submission after 26 October, as the briefing requires

Item	Disposition
Course administration (#257)	Canvas checks for the progress-report receipt, peer assessment and the final upload; owned by Guruprasath Gopal

2.3 Project Metrics

Milestones: planned against actual.

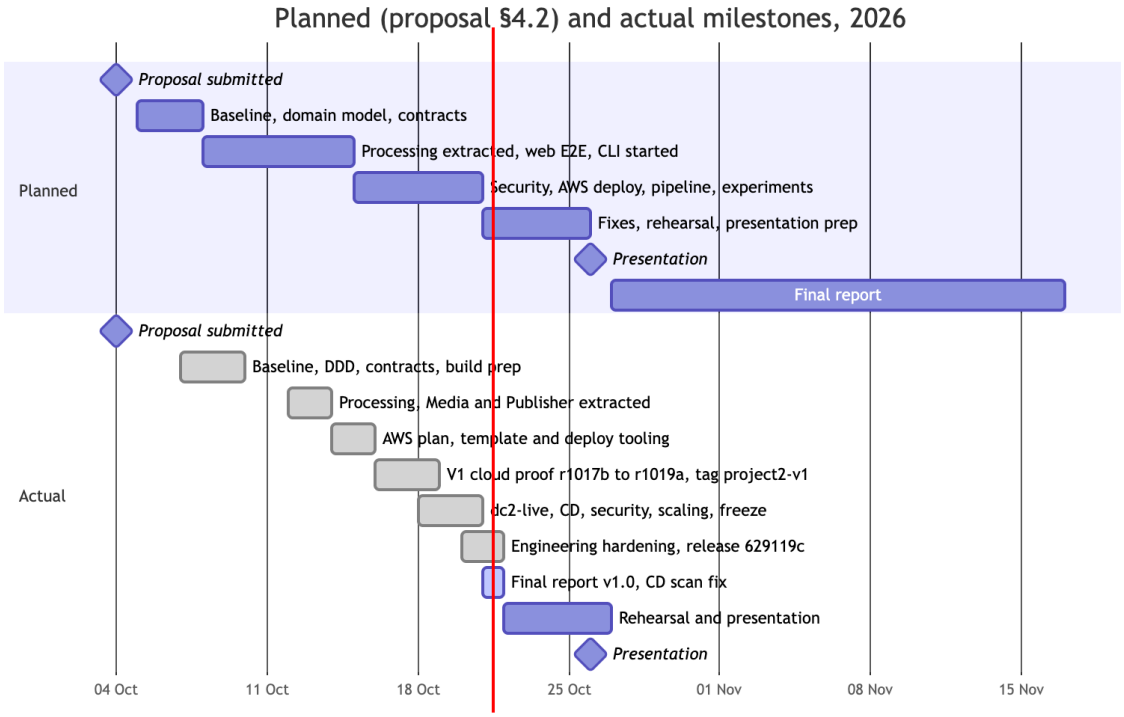


Figure 2. Planned (proposal §4.2) and actual milestones.

Planned (proposal §4.2)	Milestone	Actual	Evidence
4 Oct	Proposal submitted	4 Oct	`Team17-Proposal.docx`
5–7 Oct	Baseline frozen; domain model and contracts drafted; lecturer review	7–9 Oct (lecturer accepted; F-01, D-01 to D-07)	`project-roadmap/records/project-decisions.txt`
8–14 Oct	Processing extracted with its own data; web app end to end; CLI started	Processing 12 Oct; Media and Publisher 13 Oct (PRs #28–#30)	`processing/`, `media/`, `publisher/`
15–20 Oct	Security controls; AWS deployment and pipeline; experiments	AWS template and deploy tooling 15 Oct; CD workflow 18–19 Oct; cloud proof `r1017b` 17 Oct, `r1018b` and `r1019a` 18 Oct; tag `project2-v1` 18 Oct; `dc2-live` 18 Oct; security negatives and V2-live scaling 19 Oct; freeze 20 Oct	`docs/evidence-index.md`, `docs/release/demo-release-manifest.md`
21–25 Oct	Fixes, rehearsal, presentation preparation	Engineering release `629119c` 20 Oct; final report v1.0 and CD scan fix 21 Oct; rehearsal 22–25 Oct	`docs/release/engineering-handoff.md`; this report
26 Oct	Team 17 presentation, 14:35–15:15	Scheduled	Presentation guide p. 2
27 Oct – 16 Nov	Final report with presentation feedback	Report v1.0 complete 21 Oct; feedback revision after 26 Oct	This report

Baseline and design finished two days late, on 9 October, because the full design pack was completed before any code changed. Extraction still landed inside its planned window (12–13 October), helped by parallel integration branches. The AWS work grew from one deployment to three proof runs plus a persistent environment, and still finished inside the planned 15–20 October period.

Effort per member. The team agreed to split Project 2 effort **equally** across the three members (effort ledger, issue #191). The allocation below maps the proposal's work packages onto that split; it is not a timesheet. One person-day is 8 hours. GG is Guruprasath Gopal, IM is Ibrahim Mammadov and NKL is Nguyễn Kim Long.

#	Work package	Planned	GG	IM	NKL	Actual
1	Baseline, reproducible setup, inherited scan fix	2	2			2
2	Domain analysis, contracts, OpenAPI	3			3	3
3	Extract Processing (database, migrations, worker, recovery)	6		5		5
4	Adapt Media (asset records, Processing integration, retry)	3		3		3
5	Boundary security	3		2		2
6	Publisher CLI and contract tests	2			2	2
7	AWS environment, per-service deployment, CI/CD (deploy, release, rollback), smoke	4	6			6
8	Experiments: scaling, load, recovery, playback	3			3	3
9	Integration fixes and re-tests (close-out #265–#273)	2	2		1	3
10	Progress report, presentation and final report (to 20 Oct)	3	1	1	1	3
11	Release freeze and evidence index (not in the proposal)	0			1	1
	Recorded to 20 October	**31**	**11**	**11**	**11**	**33**
10	Remaining: rehearsal, presentation and feedback revision (forecast)		1	1	1	3
	Total at submission (forecast)	**31**	**12**	**12**	**12**	**36**

Total effort.

- **Recorded to 20 October:** 33 pd (264 h), 11 pd (88 h) per member. This is 2 pd over the 31 pd estimate, because the CD workflow (#178) and the close-out fixes (CD script bugs, Aurora pause, queue drain) were added.
- **Forecast at submission:** 36 pd (288 h), 12 pd (96 h) per member, about 16 % over the estimate.
- **Project 1 effort.** Each member's Project 1 effort is reported with their own Project 1. For Ibrahim Mammadov, that is FlickPond itself. The inherited FlickPond capabilities are credited to the FlickPond team and are not counted here.

Delivery metrics (GitHub and CI, 21 October). All commits, pull requests and issues in the repository were made through one GitHub account, `Collaboration95`, which owns the

repository. The git history therefore cannot attribute work to individual members; the effort table above is the attribution of record.

Metric	Value
Pull requests	128 opened, **123 merged** over 9 days (12–21 Oct): 27 into `Main`, 42 into `dev/closeout`, 39 into `dev/v1-integration`, 4 into `dev/v1`, 11 into short-lived `work/*` and `quality/*` integration branches
Issues	**156** created (12–19 Oct); 147 closed, 9 open (deck, report, recordings, Canvas checks, audit index, two optional V2 items)
Project board	118 items: 109 Done, 8 Todo, 1 In Progress
Commits on `Main`	533 (140 merge commits), 28 September – 21 October
CI runs	246 runs of the CI workflow: 201 pull request, 32 push, 12 manual, 1 weekly
Automated tests	3,982 Python tests collected (2,452 test functions in 197 files) plus 177 frontend tests; full offline regression 3,697 passed **[Local]**
Coverage	Media 93.91 %, Processing 93.90 % (statements plus branches); all packages pass the 80 % floor; frontend 96.24 % of statements
Code size	Service and client code 10,062 lines of Python (Media, Processing, engine, contracts, delivery, runtime support, Publisher) plus 1,822 lines of frontend JavaScript; operator and CI tooling 42,075 lines; tests 54,011 lines

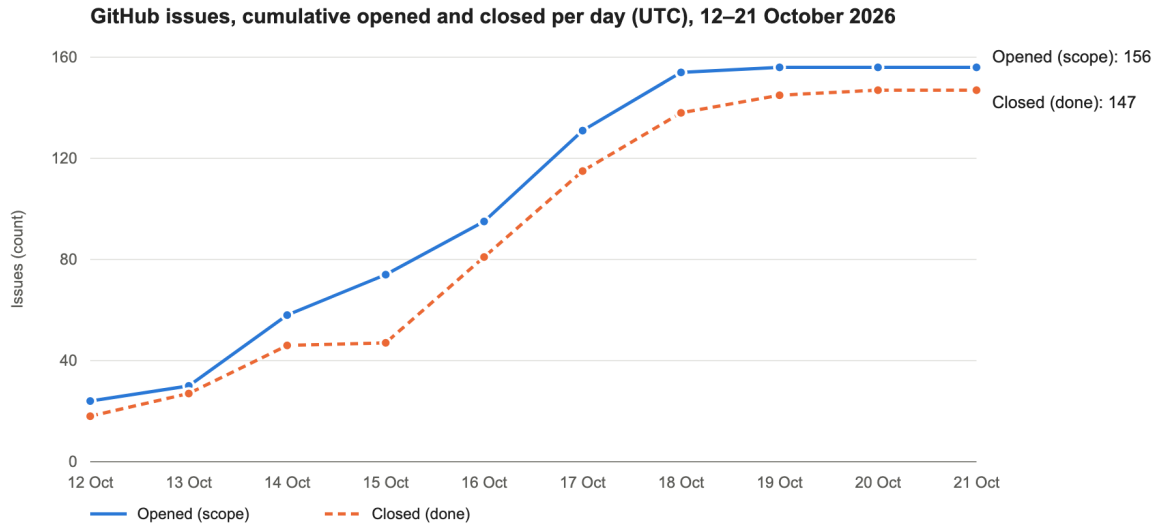


Figure 3. Issue burn-up derived from GitHub issue timestamps: scope (opened) grew in steps as the AWS readiness and close-out backlogs were added, and closure tracked it within one to two days.

The burn-up counts issues, not estimates; the issues carry no story points. The scope steps on 14–15 and 17 October are the AWS readiness backlog and the close-out backlog. The nine open issues are submission tasks and two optional experiments.

3. Solution Overview

This section describes the solution **as built** at the delivered engineering source. The proposal's Figures 2–3 (synchronous `SubmitJob`, Redis/RQ, a service token) are the "before" design. Where the build changed a proposal decision, the change and its reason are recorded in the decision register (`docs/architecture/decisions.md`) and summarised here. Decision identifiers AD-01 to AD-28 are the register's own. They are kept unchanged across sections 3.1.1, 3.2.1 and 3.3, rather than renumbered, so every reference in the repository still resolves.

Evidence labels used in this report:

- **[AWS]**: recorded in a live cloud receipt.
- **[Local]**: tests on real PostgreSQL 16 and FFmpeg, with Moto standing in for S3 and SQS.
- **[Code]**: read from source or the rendered CloudFormation template.
- **[NOT_RUN]**: implemented, never run against AWS.

3.1 Logical Architecture & Design

The architecturally significant elements, their types and their providers are listed below. “Team 17” marks custom code written in this project. “Inherited” marks code reshaped from the FlickPond baseline.

The logical view (Figure 4) groups these elements into tiers. The two domain services each have their own API, domain and persistence layers. They share only the published contracts and the delivery helpers, which are copied into each image at build time.

Element	Type	Provider	Responsibility
FlickPond web app (`frontend/`)	Custom browser SPA (plain JavaScript, vendored hls.js and Plyr)	Team 17, adapted from inherited UI	Register, log in, direct upload, status polling, MP4/HLS playback, retry, edit, delete
Publisher CLI (`publisher/`)	Custom Python client (stdlib plus `httpx`)	Team 17 (new)	Bulk publish, status, output and retry through the public HTTPS API only
Media Management (`media/`)	Custom FastAPI service and Lambda handlers	Team 17	Accounts, ownership, upload grants and verification, the public job view, playback signing, deletion
Media authorizer (`media/authorizer.py`)	Custom Lambda REQUEST authorizer	Team 17 (part of Media)	Verifies the session cookie against the current user row; returns the trusted actor
Processing (`processing/`)	Custom FastAPI service and Lambda handlers	Team 17	Jobs, attempts, leases, retry and cancel, output descriptors, recovery, encoder launch
Finite encoder (`processing/worker/`, `media_engine/`)	Custom container task	Team 17; FFmpeg command builders inherited and reshaped	Claims one attempt, verifies the pinned source, encodes MP4 and the HLS ladder, writes outputs
Shared kernel (`contracts/`, `delivery/`, `runtime_support/`)	Custom libraries copied into both images	Team 17	Versioned envelope and actor contracts, outbox/inbox and SQS adapters, Lambda DB sessions and settings
Operator and delivery tooling (`scripts/aws/v1/`, `scripts/ci/`)	Custom Python CLIs and shell	Team 17	Template rendering, deploy, session, release and restore, evidence harnesses, CD admission
FFmpeg and ffprobe	COTS open source	FFmpeg project (Debian package)	Probing and transcoding
FastAPI, Pydantic, SQLAlchemy, Alembic, asyncpg, boto3, PyJWT, bcrypt	Open-source libraries	Python community	Web framework, validation, ORM, migrations, PostgreSQL driver, AWS SDK, JWT, password hashing
Relational database (PostgreSQL 16)	Managed service	AWS (Aurora Serverless v2)	`media_db` and `processing_db`
Object storage	Managed service	AWS (S3)	Source, output and frontend buckets
Messaging	Managed service	AWS (SQS)	Three queues and three dead-letter queues

Element	Type	Provider	Responsibility
Edge, compute and operations	Managed services	AWS (CloudFront, API Gateway, Lambda, ECS Fargate, EventBridge, SSM, Secrets Manager, ECR, CloudWatch, Budgets, IAM)	Section 3.2
Source control, CI/CD and scanners	SaaS and open-source tools	GitHub (Actions, OIDC); Aqua Trivy, gitleaks, OWASP ZAP, Ruff	Section 5

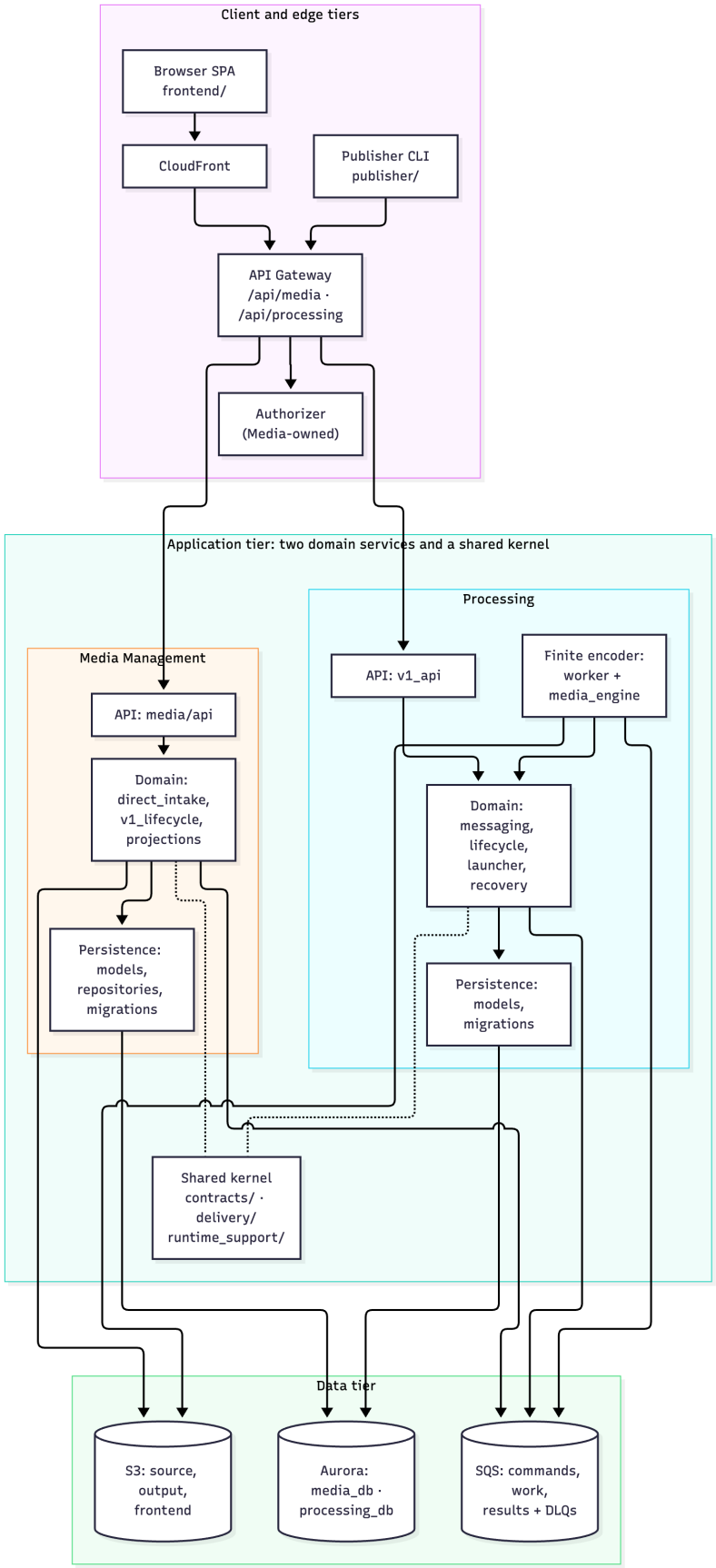


Figure 4. Logical view: client and edge tiers, the two domain services with their API, domain and persistence layers, the shared kernel, and the data tier.

The relationships that matter most are the ones that **do not** exist:

- Media never calls Processing, and Processing never calls Media. Every interaction is a message on SQS, or a read of the context's own database.
- Neither service holds the other's database credentials. Each Lambda role can read only its own domain's connection string.
- The worker cannot send any message. It reports progress only through the Processing database, and the Processing outbox publishes the results.

Figures 5, 6 and 7 show the dynamic relationships for the core workflow (UC-1 to UC-3), split into its three phases. Section 3.2.4 explains each step.

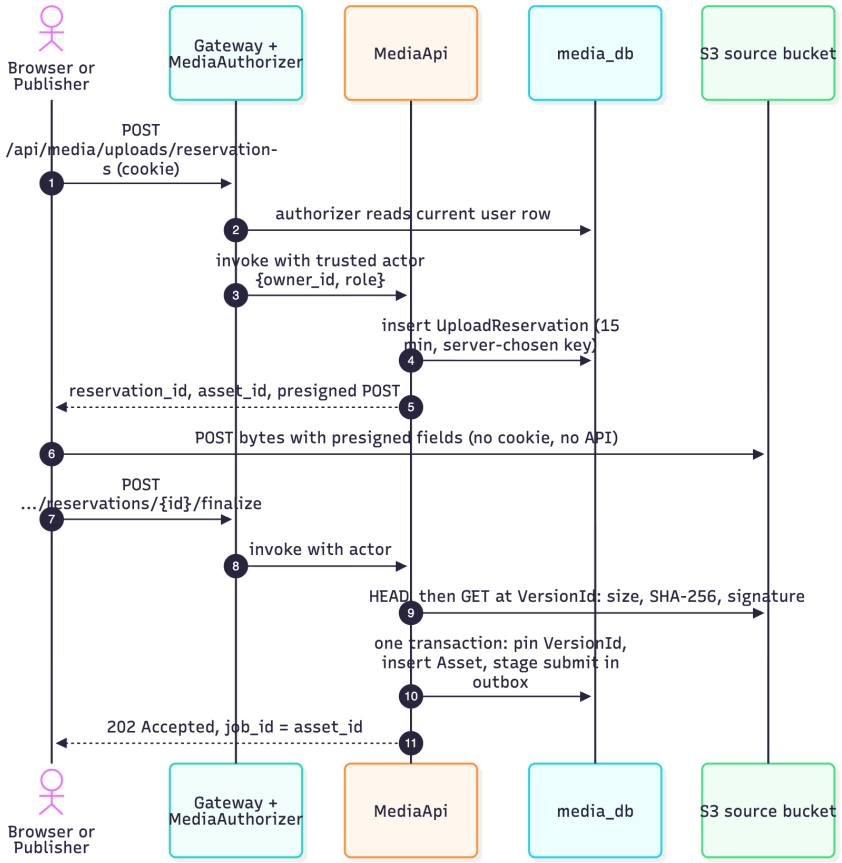


Figure 5. Dynamic view, phase 1, intake: reserve, direct upload to S3, verify and adopt.

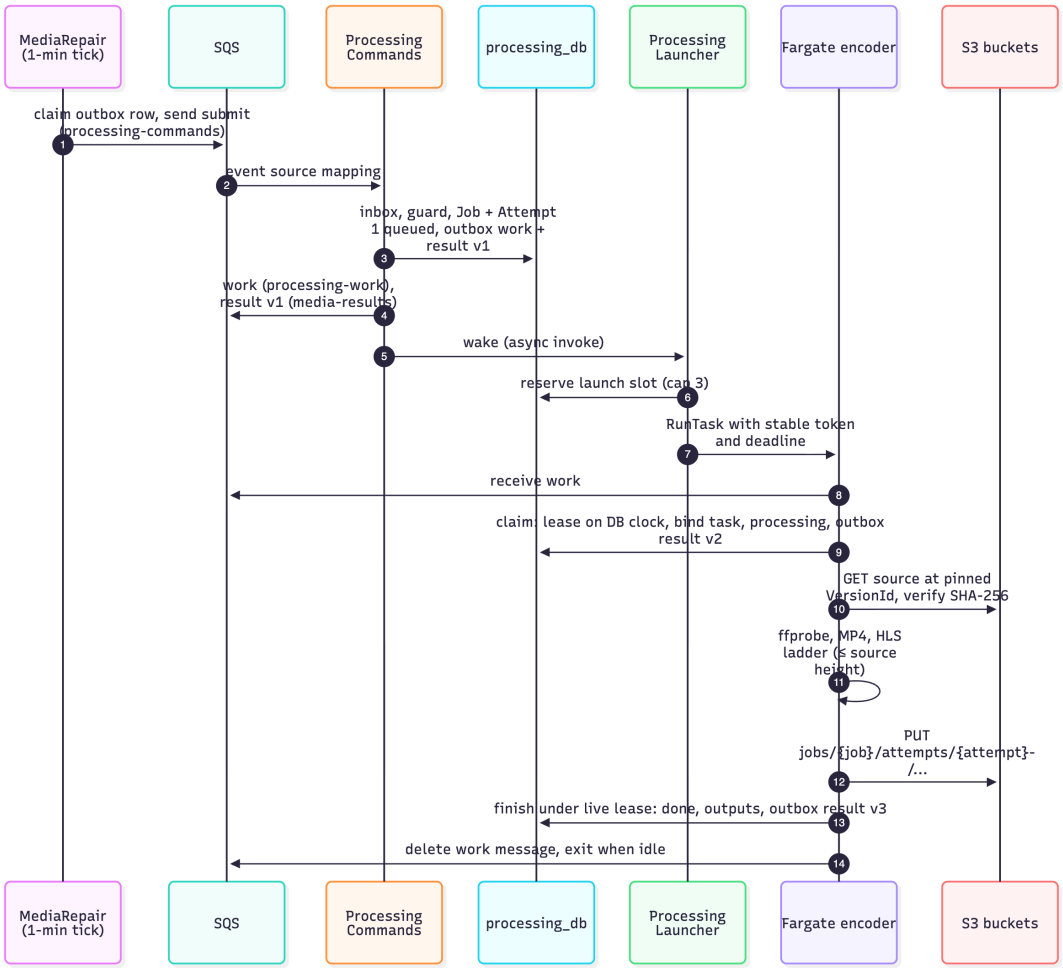


Figure 6. Dynamic view, phase 2, handoff and encoding: outbox → SQS → Processing inbox, launch, lease, encode, finish.

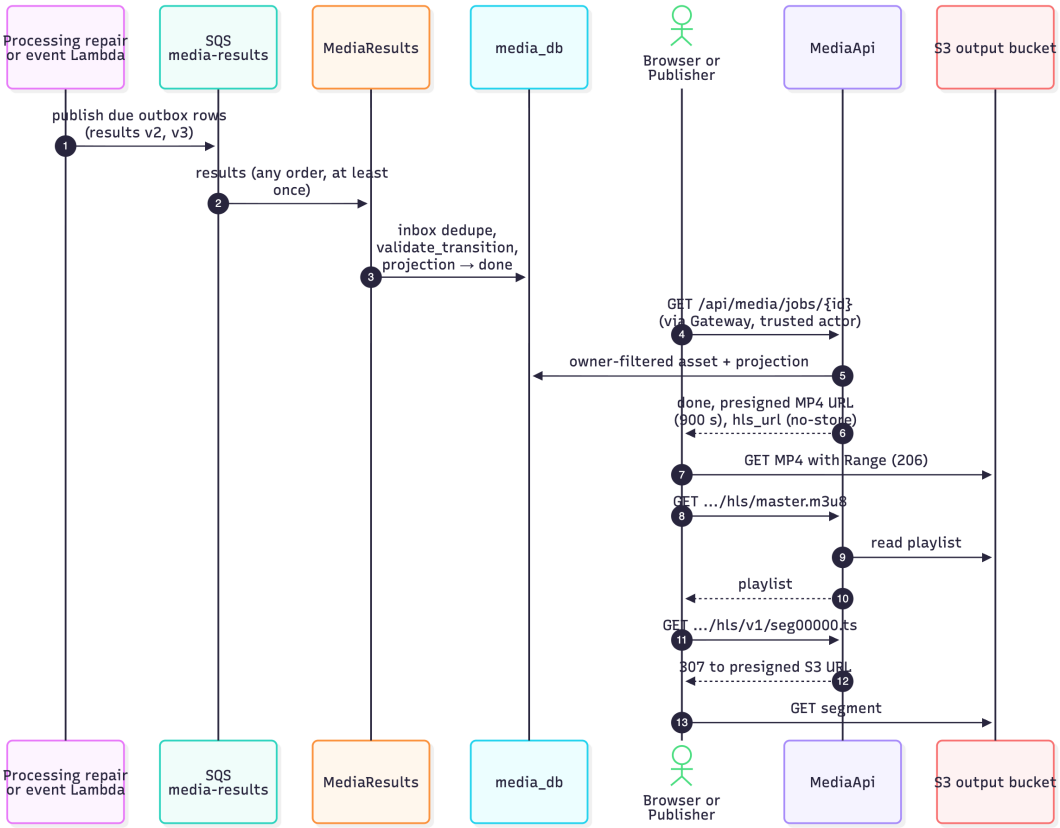


Figure 7. Dynamic view, phase 3, results and playback: Processing outbox → Media projection; owner-checked, signed playback.

3.1.1 Key Architectural Decisions

The significant decisions reflected in the logical view are summarised below. Each has a full record in the register (context, decision, alternatives, trade-off, evidence).

Identifier	Decision	Rationale and trade-off	Status
AD-01	Extract **Processing** as the only new business service; Identity and Delivery stay inside Media	Processing has its own rules (claim, lease, attempt), lifecycle and CPU-bound scaling profile. Every playback or retry decision needs owner and asset together, so splitting Identity or Delivery out would turn single-database joins into cross-service calls. Four services were rejected as more contracts and operations than the need justifies.	Kept, confirmed by detailed DDD (F-01)
AD-02	**Database per service**: `media_db` and `processing_db`, separate migration histories and roles (`*_runtime` DML only, `*_migrator` DDL, `*_owner` no login)	A real ownership boundary at low cost. Trade-off: both databases share one Aurora cluster, so logical ownership is real but failure isolation is not (AISS-01).	Kept (hosting refined by AD-12)
AD-09	**API Gateway HTTP API with a Media-owned Lambda authorizer**; the Gateway adapter builds the actor only from the authorizer context	Processing trusts the owner identity without holding Media's JWT secret or user table. Result cache TTL is 0, so a deleted user or changed role takes effect at once. The cost is a database read per request.	New (supersedes AD-06 service token)

Identifier	Decision	Rationale and trade-off	Status
AD-11	**SQS with an SQL outbox and inbox** in both databases, replacing Redis/RQ and synchronous `SubmitJob`	No always-on broker, and failure isolation: Media accepts uploads while Processing is down. The SQL-to-SQS handoff is not atomic, so repair is part of the design. Delivery is at least once, so duplicates are normal and consumers deduplicate.	New (supersedes AD-03; changes AD-04)
AD-14	**Direct-to-S3 intake**: a presigned, owner-scoped reservation, then a finalize step that verifies size, SHA-256 and file signature at a pinned S3 VersionId	No 100 MiB body passes through Gateway or Lambda, and the bytes that were verified are exactly the bytes encoded. Trade-off: a two-step protocol; the client hashes the file first, and versioned buckets need cleanup.	New (changes AD-05)
AD-21	**Media-owned projection** of Processing state, fenced by `state_version` and attempt	Media answers status and signs playback without querying Processing. The view is eventually consistent; stale or reordered results are ignored.	New
AD-24	**`contracts/` as published language and shared kernel**, versioned N / N-1	Additive changes ship in either release order; consumers accept the current and previous versions and ignore unknown fields. Still a shared kernel: a breaking change needs two releases.	New
AD-26	Retired peer-HTTP and Redis/RQ code **quarantined in `legacy/`** and excluded from images by a build-time check	The submission shows no dead dependency between the services. Trade-off: the legacy package and its CI lane remain to maintain.	New

3.1.2 Tiers and Layers

Tier	Elements	Layers and modules
Client	Browser SPA, Publisher CLI	Presentation only. Both call the same public API. The browser uploads bytes straight to S3 with a presigned POST.
Edge	CloudFront, API Gateway, Media authorizer	TLS termination, routing (`/api/media/*`, `/api/processing/*`), identity, stage throttle
Application	Media Management, Processing	**API layer**: `media/api/*`, `processing/v1_api.py`; transport is FastAPI behind `contracts/gateway.py`. **Domain and service layer**: `media/services/direct_intake.py`, `v1_lifecycle.py`, `media/projections.py`; `processing/messaging.py`, `lifecycle.py`, `launcher.py`. **Messaging layer**: `media/messaging.py`, `processing/messaging.py` over `delivery/`. **Persistence layer**: models, repositories and Alembic migrations per service. **Shared kernel**: `contracts/`, `delivery/`, `runtime_support/`.
Execution	Lambda handlers, Fargate encoder	Queue consumers, one-minute repair ticks, ECS and S3 event handlers, the launcher, and the encoder (`processing/worker/` with `media_engine/`)
Data	Aurora, S3, SQS	`media_db`, `processing_db`; source, output and frontend buckets; commands, work and results queues

The dependency rules are enforced, not just documented:

- Processing's API module imports no Media package.
- `deploy/v1/check_runtime.py` fails an image build if the opposite domain package, or any `legacy/` module, can be imported.
- The engine package is pure command construction with no I/O.

3.1.3 Nodes and Subsystems

Subsystem	Responsibility	Nodes	Owner
Identity and intake	Accounts, JWT cookie, upload reservations, verified adoption of uploads	`MediaApi`, `MediaAuthorizer` Lambdas	Media
Playback	Owner-checked presigned MP4 URLs, HLS playlist and segment redirects	`MediaApi`	Media
Projection	Apply Processing results to Media's read model	`MediaResults` Lambda	Media
Media repair	Publish the outbox, expire grants, clean up deleted sources	`MediaRepair` Lambda (one-minute rule)	Media
Job orchestration	Job and attempt lifecycle, retry, cancel, leases, state versions	`ProcessingApi`, `ProcessingCommands` Lambdas	Processing
Recovery	Rebuild lost work, expire leases, react to stopped tasks and late outputs	`ProcessingRepair`, `ProcessingEcsEvents`, `ProcessingS3Events` Lambdas	Processing
Capacity	Start finite encoders from the SQL backlog, at most three	`ProcessingLauncher` Lambda (V2-live)	Processing
Encoding	Probe, MP4 or edited output, HLS ladder, upload outputs with SHA-256	Fargate encoder task (2 vCPU, 4 GiB), 0 to 3	Processing
Messaging	Envelope validation, outbox, inbox, SQS lanes, DLQ tooling	Library in both images	Shared kernel
Operations	Template, deploy, release and restore, evidence	CD runner or operator station	Platform

Media and Processing are the only business subsystems. The Lambdas, the encoder and the repair handlers are **execution roles** of one context. Each runs that context's code under that context's image, database role and IAM role. They are not extra services, because none has its own data, rules or release unit.

3.1.4 Platform Design

The platform design comes from domain-driven design (`docs/architecture/ddd.md`). The proposal drew the boundary from the language of the workflow (*upload, submit, claim, encode, complete, play*). The detailed analysis requested in lecturer feedback F-01 then checked that boundary against the code's real invariants and transaction boundaries.

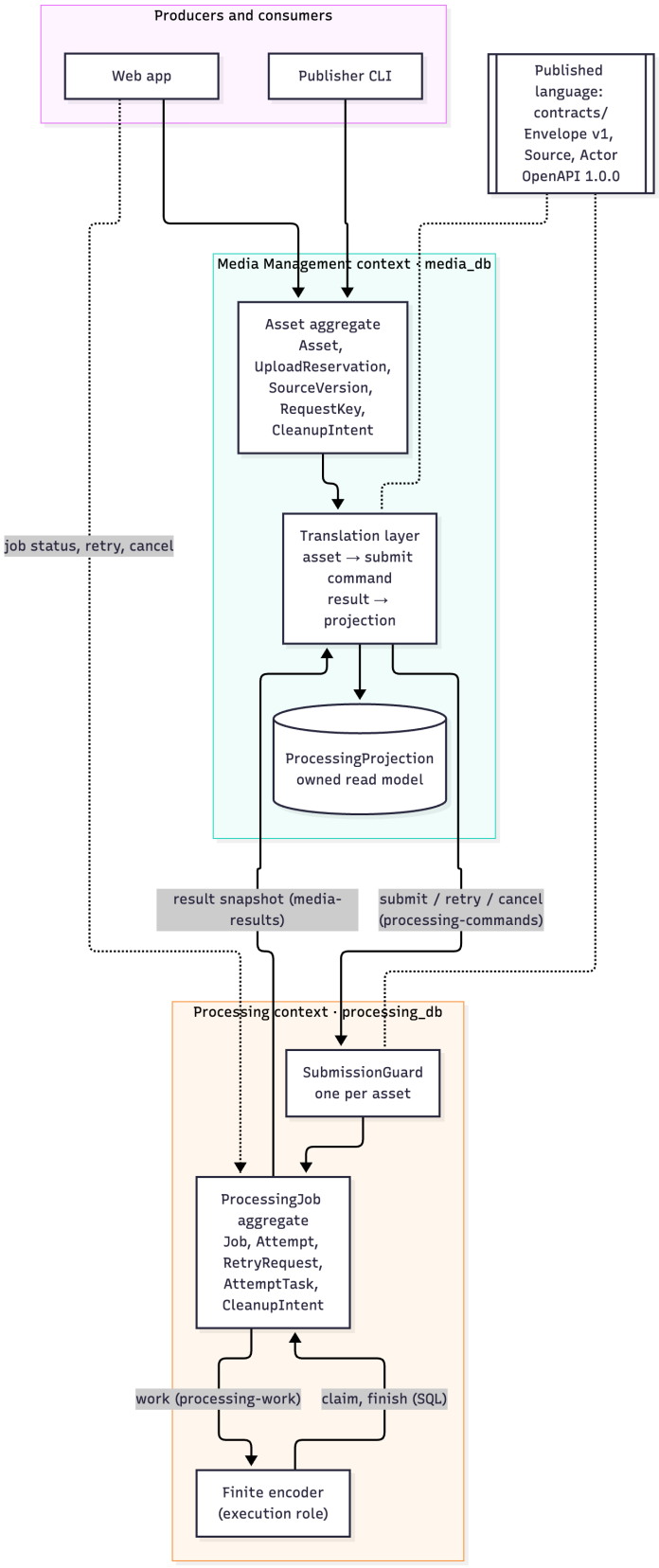


Figure 8. Context map: the Media Management and Processing contexts, their aggregates, the translation layer and the published language.

Bounded contexts.

Context	Owns	Does not own
Media Management	Accounts and login, ownership, upload intake, source versions, playback signing, the public job view, deletion and source cleanup (`media_db`)	Job state, attempts, leases and outputs. It holds only a projection.
Processing	Jobs, attempts, leases, retry and cancel, output descriptors, late-output cleanup, encoder claims and launch reservations (`processing_db`)	Users, assets, source bytes (read-only S3 grant) and playback links

Ubiquitous language (selected).

- **Owner**: the only principal that may read, retry, edit, delete or play an asset. Processing holds the owner as a plain UUID, never a foreign key.
- **Asset**: an owned video. Its UUID is also the public `job_id`.
- **Upload reservation**: a 15-minute, owner-scoped grant to write one S3 object.
- **Source revision**: the immutable bytes Processing may read, identified by the pinned S3 VersionId plus SHA-256 and size.
- **ProcessingJob**: exactly one per asset.
- **Attempt**: one numbered try, never reused.
- **Lease**: one worker's time-limited right to finish an attempt, judged on the database clock.
- **Output descriptor**: the kind, key, size and SHA-256 of an output.
- **Command, work and result**: the three message families (below).
- **Projection**: Media's own copy of Processing state.

Aggregates and their invariants. The **Asset** aggregate (root `Asset`, with `UploadReservation`, `SourceVersion`, `RequestKey` and `CleanupIntent`) enforces these rules:

- Only the owner can read or change an asset; anyone else gets 404.
- A reservation is idempotent per owner and request ID.
- The stored bytes must match the declared size, SHA-256 and signature.
- An adopted source cannot change.
- Adopting a source and staging its `submit` command happen in one commit.
- Retry is allowed only from `failed`.
- Delete is a tombstone plus a `cancel` command in one transaction.

The **ProcessingJob** aggregate (root `Job`, with `Attempt`, `RetryRequest`, `AttemptTask` and `SubmissionGuard`) enforces these rules:

- There is one job per asset; an identical repeat submit is a no-op.
- Commands must match the stored owner, asset, job and source.
- Transitions are one-way per attempt. A job returns to `queued` only through a **new** attempt.
- A worker needs a valid lease to start and to finish, and one task claims at most one attempt.
- Cancel before submit is permanent.
- `state_version` only increases, and every state change commits together with its outgoing messages.

Both aggregates are consistency boundaries. No transaction spans the two databases.

Commands, results and domain events. All five message kinds share one versioned envelope (`contracts/v1.py`): `schema_version`, `message_id`, `kind`, owner, asset, job and attempt IDs, `state_version`, an immutable `source` descriptor, and a payload. Payload values that look like secrets or URLs are rejected, so queues never carry bytes, credentials or signed URLs.

Message	Queue	Producer	Consumer	Purpose
`submit`	`processing-commands`	Media outbox (sent by `MediaRepair`)	`ProcessingCommands`	Process a newly adopted asset
`retry`	`processing-commands`	Media	`ProcessingCommands`	New attempt over the stored source
`cancel`	`processing-commands`	Media	`ProcessingCommands`	Tombstone; may precede `submit`
`work`	`processing-work`	Processing outbox	Fargate encoder	Wake an encoder for one attempt
`result`	`media-results`	Processing outbox	`MediaResults`	Full job state snapshot at a `state_version`

The system has no separately named domain events such as `JobStarted`, and it is **not** event sourced. Each claim, completion, failure, retry or cancel emits one `result` snapshot with a monotonically increasing `state_version` (event-carried state transfer). Producer authority is not part of the schema. Queue resource policies and per-handler IAM roles enforce it: a direct `SendMessage` from any other principal was refused on all three queues **[AWS]**.

Context-map relationships.

- **Customer/supplier.** Media is the customer: it states what to process. Processing is the supplier and owns the answer.
- **Translation on both sides.** Media builds a `submit` envelope from its asset and pinned source, and turns each result into its own projection and public status. Processing turns each command into `Job`, `Attempt` and `SubmissionGuard` rows.
- **Published language.** `contracts/v1.py`, `contracts/gateway.py`, the two OpenAPI 1.0.0 documents, and frozen fixtures that a contract change must update in the same pull request.

Interaction with producers and consumers. The platform roles from the briefing map onto the design as follows.

Platform role	As built	How it interacts
Seed	An owned video asset and its processing job (`Asset` plus `Job`, linked by `asset_id`)	Created by finalize; the single shared identifier across contexts
Producers	The web app (one file at a time) and the Publisher CLI (batches)	Public API only: reserve → direct S3 POST → finalize; status, retry, edit, delete
Consumers	Owners playing their own output	Presigned MP4 with range requests; HLS playlist through Media; segments by 307 redirect to presigned S3 URLs
Integrators	Developers building new clients	Two OpenAPI documents, `contracts/`, and the Publisher as reference client. The Publisher has 0 imports from service code and 0 database credentials, enforced by a test.
Operators	The team	CD workflow, release and restore CLI, evidence harnesses; V2-live launches encoders automatically

3.2 Physical Architecture & Design

The platform runs in one AWS account, in region **ap-southeast-1 (Singapore)**. One CloudFormation stack, rendered by `scripts/aws/v1/definitions.py` and checked in as `deploy/aws/v1/template.json`, defines all of it. The checked-in template is tested to be identical to the rendered output.

Two operating modes used the same template:

- **V1 (10–18 October, tag `project2-v1`).** One stack per proof run (`r1017b`, `r1018b`, `r1019a`). Each stack was closed outside booked sessions and deleted afterwards.
- **V2-live (18 October to the end of the course).** One persistent stack, `dc2-live`. It is always open, bounded by a throttle and a cost budget, and updated in place by CD after every green CI run on `Main`. It scales to zero when idle (AD-27).

Figure 9 shows the persistent topology. The network rules and data paths are listed after the component table.

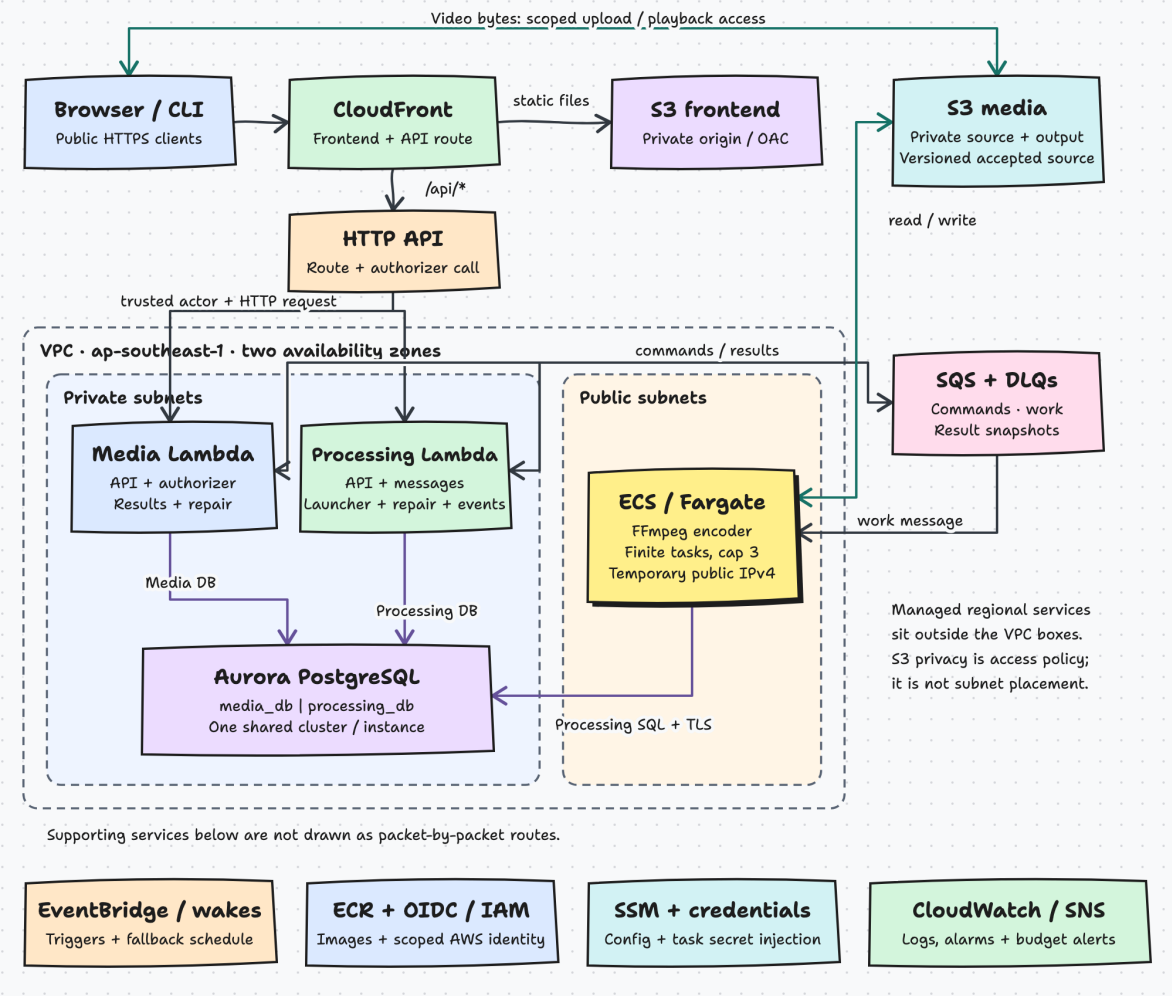


Figure 9. Persistent V2-live AWS topology (ap-southeast-1): public edge, VPC-attached Lambdas, finite Fargate encoders, Aurora, S3 and SQS.

Component	Deployed as	Placement and sizing
Web app	Static files in a private S3 bucket behind CloudFront (origin access control)	Global edge, HTTPS only (HTTP redirects); content-hashed assets
Public API	One API Gateway HTTP API: `/api/media/*`, `/api/processing/*`, one REQUEST authorizer	Regional; V2-live stage throttle 20 requests/s, burst 50
Media	One container image → 4 Lambda functions (`MediaApi` 2048 MB, `MediaAuthorizer`, `MediaResults`, `MediaRepair`), each with a `live` alias	Private subnets in two AZs, VPC-attached
Processing	One container image → 6 Lambda functions in V2-live (`ProcessingApi`, `ProcessingCommands`, `ProcessingRepair`, `ProcessingEcsEvents`, `ProcessingS3Events`, `ProcessingLauncher`)	Private subnets in two AZs
Encoder	Worker container image as an ECS Fargate task definition; no ECS service	2 vCPU, 4 GiB; 0 when idle, at most 3; public subnet with a temporary public IPv4 while running
Databases	Aurora PostgreSQL 16 Serverless v2: one cluster, one instance, `media_db` and `processing_db`	Private subnets; 0–2 ACU, auto-pause when idle; encrypted; `rds.force_ssl=1`
Object storage	Three S3 buckets: source, output, frontend	Versioned, SSE-S3, public access blocked, TLS-only policies
Messaging	Three Standard SQS queues and three DLQs	SQS-managed encryption, TLS-only, role-scoped resource policies
Scheduling	EventBridge: one-minute Media and Processing repair ticks, ECS task `STOPPED`, S3 `Object Created`	Regional
Configuration	SSM Parameter Store SecureStrings (four role DSNs, JWT secret); RDS-managed master secret	Each handler role reads only its own domain's parameters
Images	Three ECR repositories (media, processing, worker)	Immutable tags, scan on push; releases address images by digest
Observability	CloudWatch log groups (7 days); 13 alarms (one per queue DLQ, one per function); `ServicesDashboard`; AWS Budget USD 20/month at 50/80/100 %	Per service

Network.

- One VPC (`10.62.0.0/16` plus an Amazon IPv6 block) spans two Availability Zones, with one private and one public subnet in each.
- Private subnets have an IPv6 route through an egress-only internet gateway, **no IPv4 default route and no NAT gateway**. Lambdas reach SQS, SSM and ECS through dual-stack endpoints.
- One S3 gateway endpoint, attached to all route tables, is restricted to the source and output buckets.
- No security group has an inbound rule except the database's, which admits only the Lambda, worker and migration groups on port 5432.
- An external scan of the public surface found only ports 80 and 443 open, and 80 redirects to HTTPS **[AWS]**.

3.2.1 Key Architectural Decisions

These decisions are reflected in the physical view. Section 3.1.1 holds the logical ones.

Identifier	Decision	Rationale and trade-off	Status
AD-10	**Lambda container images, one image per domain**; `ImageConfig.Command` picks the handler; every function has a `live` alias	Scale-to-zero compute with a per-domain release unit; FastAPI code runs unchanged behind an adapter. Trade-offs: cold starts, no pooled connections (per-invocation `NullPool`, so a paused database can stay paused), a 29 s Gateway timeout.	New (changes AD-07)
AD-12	**Aurora Serverless v2**: one cluster, one instance, two logical databases, separate roles; 0–2 ACU with auto-pause	Near-zero idle cost and real role separation. Trade-off: one shared capacity and failure boundary; no replica failover; a request to either domain resumes the cluster (first read after a pause: 4.7 s **[AWS]**).	New (refines AD-02)
AD-13	**Finite Fargate encoders**: one task per attempt, a hard cap of 3, zero when idle	FFmpeg needs minutes of CPU, and nothing should run idle. Each task claims at most one attempt and exits after two empty polls. Alternatives rejected: an always-on ECS service with autoscaling (idle cost) and AWS Batch (another dependency). Trade-off: at most three concurrent encodes; Fargate start time adds to queue wait.	New
AD-15	**CloudFront as the single public entry** for frontend and `/api/*`	Managed HTTPS without buying a domain; one origin for cookies. Trade-off: AWS-generated hostnames; the `execute-api` endpoint is also reachable directly (AISS-06).	New
AD-16	**Per-owner release and rollback by image digest and Lambda alias**	Releases Processing without touching Media (UC-7). Each release builds and scans the owner's image, publishes versions, moves `live` aliases, runs a smoke test, and restores the previous versions on failure. Trade-off: rollback is image-only, and schema changes must be additive.	New
AD-17	**GitHub Actions CD over OIDC**: role `dc2-github-deploy`, assumed only from the `aws` environment, with no stored AWS key	Recorded, repeatable delivery. Trade-off: the role is broad (`AdministratorAccess` with a region lock), protected by the environment's branch policy (AISS-09).	New
AD-18	**Private Lambda subnets with IPv6 egress only, an S3 gateway endpoint and no NAT**	Avoids NAT and interface-endpoint hourly charges, the largest idle cost in a typical VPC design. Trade-off: every AWS dependency must be reachable over IPv6 or the gateway endpoint; Fargate tasks run in public subnets to pull images.	New
AD-20	**Least-privilege IAM per handler**, native role credentials, queue resource policies	Producer and consumer authority is enforced by AWS, not by the message schema. `ProcessingApi` has no S3 or SQS access; `MediaApi` has no SQS access; the worker can read sources, write outputs and receive work, nothing else. Trade-off: the worker's source read is bucket-wide, so owner binding is checked in code before any read.	New (supersedes AD-06)
AD-23	**One CloudFormation stack, applied in two phases** (infrastructure, then applications pinned by digest)	The smallest operational surface for three people. Trade-off: infrastructure for both domains changes together; only code releases are independent.	New

Identifier	Decision	Rationale and trade-off	Status
AD-25	**Per-service alarms and one services dashboard**	Failures are attributable to one service: one DLQ alarm per queue and one error alarm per function, tagged by owner. Trade-off: alarms are state to read, with no paging target.	New
AD-27	**One persistent, always-on environment, updated in place, scaling to zero**	Per-run stacks cost a 25-minute deploy and a 25–40-minute teardown for every check. The stack already costs about USD 3–5 a month idle, so it now stays up, bounded by the throttle, the encoder cap and a USD 20 budget. Encoders start automatically from the SQL backlog. Trade-off: the environment is public all the time, and old and new code run side by side for a moment during a release.	New (supersedes AD-19 booked sessions)

3.2.2 Technology and Services

Technology or service	Used for	Rationale	Trade-off accepted
Python 3.12, FastAPI, Pydantic, SQLAlchemy 2 (async), Alembic	Both services, the worker and the tooling	Continuity with the inherited code; typed validation at every boundary; one migration history per database	One lockfile for the repository; owner extras keep each image to its own packages (40/36/33 exported packages)
FFmpeg and ffprobe	Probing, MP4 encoding, HLS ladder (360/480/720, capped at the source height)	Industry standard and already proven in Project 1	CPU-bound; HLS is best effort; a large unfixed-CVE surface in the worker image (section 4.3)
Plain JavaScript SPA, hls.js, Plyr	Web client	No build step; browsers load `app.js` directly; Vitest tests it	No production bundling or minification
Python stdlib plus `httpx`	Publisher CLI	Proves reuse with the smallest dependency set: 491 lines in 6 files	Polling-based status; one cookie jar per process
AWS Lambda (container images)	APIs, consumers, repair, launcher	Pay per request, scale to zero, per-domain image release	Cold starts; account concurrency limit of 10 observed
Amazon API Gateway HTTP API	Routing, authorizer hook, throttle	Cheapest managed HTTP front for Lambda; REQUEST authorizer	No WAF; one stage-level limit
Amazon CloudFront	HTTPS entry, static site, `/api/*` pass-through	Managed TLS, HTTP/2 and HTTP/3, no custom domain needed	Playback bytes come from S3 directly, not through CloudFront
Amazon SQS (Standard) with DLQs	Commands, work and results	Serverless, durable, cheap; redrive policy (5 receives)	Unordered and at least once, hence inbox dedupe and version fences
Amazon Aurora PostgreSQL Serverless v2	Two owned relational databases	Transactions, constraints and `SKIP LOCKED` for outbox claims; scales to 0 ACU	Single instance; cold resume of several seconds
Amazon S3 (versioned)	Sources, outputs, frontend	Presigned POST/GET keep bytes off the API; versioning makes the pinned source immutable	Cleanup must delete every version
Amazon ECS on Fargate	Finite encoders, migrations, bootstrap task	No hosts to manage; per-task CPU caps make capacity predictable	Task start median 25.6 s **[AWS]**, which dominates queue wait
Amazon EventBridge	Repair ticks, ECS and S3 event hints	Rebuild lost work and react to stopped tasks without polling clients	One-minute granularity
AWS IAM, SSM Parameter Store, Secrets Manager	Per-handler roles, DSNs and JWT secret, Aurora master secret	Secrets stay out of code, images and the template	SSM read at cold start

Technology or service	Used for	Rationale	Trade-off accepted
Amazon ECR	Three image repositories	Immutable tags and digests are the release identity	Scan on push complements the CI and CD Trivy scans
Amazon CloudWatch, AWS Budgets	Logs, alarms, dashboard, cost alerts	Per-service observability at near-zero cost	7-day log retention; no paging
GitHub Actions, OIDC, Pages	CI, CD, study-guide publication	Integrated with pull requests; no long-lived cloud keys	Hosted runner limits shape the CI lanes

Runtime and development tools are separate.

- Runtime images contain only the owning service's code and its runtime extras. `deploy/v1/check_runtime.py` verifies this.
- Development tools run only in CI and on developer machines: pytest, Ruff, Vitest, Playwright, Moto, cfn-lint, actionlint, Trivy, gitleaks and ZAP.

3.2.3 Persistence Design

Relational data. Each service owns one PostgreSQL 16 database on the shared Aurora cluster, with its own Alembic migration history (Media `0001_media` … `media_0008`; Processing `processing_0001` … `processing_0007`). No table has a foreign key into the other database. Owner and asset identifiers cross the boundary only as plain UUIDs inside messages. Figures 10 and 11 are the logical data models.

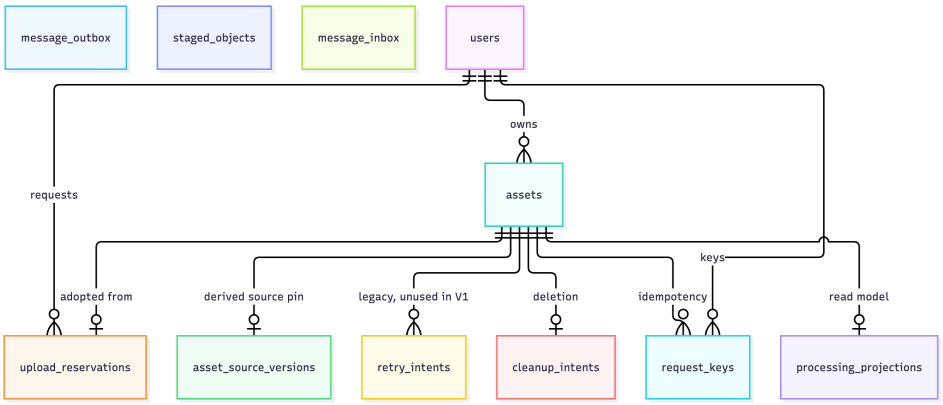


Figure 10. Media Management logical data model (`media_db`).

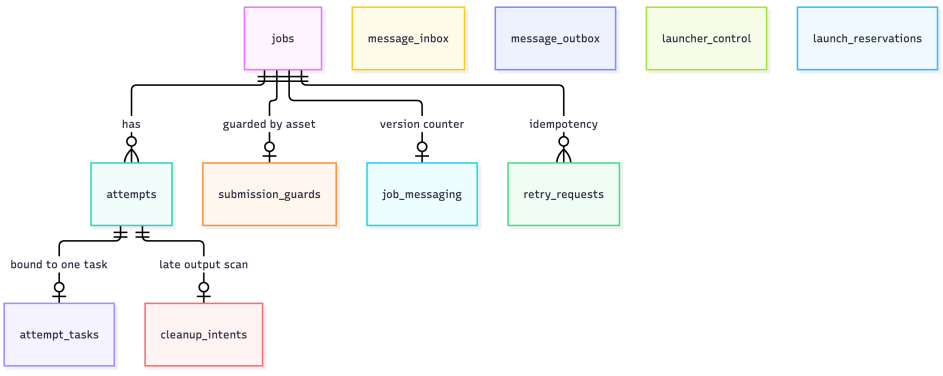


Figure 11. Processing logical data model (`processing_db`).

Database roles are created by a one-off bootstrap task:

- `*_owner` owns the database and cannot log in.
- `*_migrator` inherits the owner and runs Alembic.

- `*_runtime` has only `SELECT`, `INSERT`, `UPDATE` and `DELETE` plus sequence use.

`PUBLIC` has no access to either database, and neither domain's login roles can connect to the other database. On AWS, a runtime login to the other database was refused, and DDL as the runtime role was refused **[AWS]**. Connections require verified TLS.

Transactions and concurrency. The design's correctness rests on a few local-transaction rules:

- **State and message commit together (outbox).** Each state change stages its outgoing messages in `message_outbox` in the same transaction (`delivery/sql.py:stage`). A publisher claims due rows with `FOR UPDATE SKIP LOCKED` and a claim token, commits, sends outside the lock, and acknowledges only if the token still owns the row. A lost reply can resend the same `message_id`; it can never lose a committed message.
- **Consume once per content (inbox).** Consumers insert `message_id` plus a canonical body hash into `message_inbox` inside the domain transaction. The same ID with the same body is a no-op. The same ID with a different body is a conflict. Lambda acknowledges only after commit.
- **Serialisation per asset.** `SubmissionGuard` is locked first, then `Job`, then `Attempt`, so submit, retry and cancel for one asset never interleave. `jobs.current_attempt_id` is a deferrable foreign key, so a job and its first attempt can be inserted in one transaction.
- **Leases on the database clock.** Claim, heartbeat and finish compare `lease_token`, `lease_expires_at` and `deadline_at` with `clock_timestamp()`. Worker clocks are never trusted.
- **Monotonic projection.** `processing_projections` applies a result only if its `state_version` is newer, for the same job, source and attempt namespace. Older results are ignored, and conflicting content at the same version is rejected.

Object storage. S3 keys are an implicit contract enforced in code on both sides. Sources sit at `staging/{owner}/{reservation}/source`, or at `sources/{owner}/{asset}/source` for edit-derived assets, and are pinned by VersionId. Outputs sit at `jobs/{job}/attempts/{attempt}/primary.*` and `…/hls/`. Exactly one primary output is required, and HLS names are validated before Media serves them. Because buckets are versioned, cleanup deletes every object version, not just the latest.

What persists, and for how long. These are the values configured in the template and code. The lifecycle of this deployment as a whole is set by AD-08 (fresh demonstration data only) and AD-27 (the persistent environment is torn down once, after 16 November 2026).

Data	Persists beyond	Period configured
Session (JWT cookie)	One request	1 hour (`jwt_ttl_seconds` 3600); not stored server-side
Upload grant	One request	15 minutes; an expired reservation is rescanned for late writes and retired after a clean scan beyond a one-day horizon
Signed playback URL	One request	900 seconds (`signed_url_ttl_seconds`)
Accounts	Session	Until environment teardown. No account deletion feature or purge schedule exists

Data	Persists beyond	Period configured
Assets, sources and outputs	Session	Until the owner deletes the asset. Deletion is a tombstone; every source and output version is then deleted asynchronously. No age-based expiry and no S3 lifecycle rule are configured
Jobs, attempts and tombstone rows	Session	Until environment teardown; no hard-purge job
Outbox and inbox rows	Transaction	Until environment teardown; sent rows are kept for replay; no pruning job
SQS messages	Transaction	7 days (DLQs 14 days); visibility timeout 420 s; redrive after 5 receives
Lambda and task logs	Request	7 days
Aurora backups	n/a	1-day automated backup retention; one final snapshot is kept at teardown
CI and CD artifacts	Workflow run	7, 14 or 60 days, depending on the artifact

Information architecture: data inventory and classification. The team uses four classification levels:

- **Restricted**: credentials, keys and secrets.
- **Confidential**: personal data and owners' content.
- **Internal**: operational and processing metadata.
- **Public**: published static assets.

Account and authentication data (Confidential; Restricted for hashes and secrets)

- *Data format:* `users` rows (UUID, email, bcrypt hash at cost 10, role `user` or `operator`, timestamps). A stateless HS256 JWT in a host-only, `HttpOnly`, `Secure`, `SameSite=Strict` cookie. The signing secret is an SSM SecureString.
- *Storage technology:* PostgreSQL in `media_db`, chosen for unique constraints and transactional joins with asset ownership.
- *Security requirements and controls:*
  - Only the Media roles can read the JWT secret or the Media DSN.
  - The authorizer re-reads the current user row on every request, so role changes and deletions apply at once.
  - Password bytes are length-limited before hashing.
  - Authenticated responses use `Cache-Control: private, no-store`.
- *Trade-off considered:* A managed identity provider (Cognito) would add MFA and recovery flows, but brings a new dependency and token format. The team kept the inherited account model and moved verification to the edge (AD-09). There is no password reset, and no key rotation.

Owner media content: source videos and encoded outputs (Confidential)

- *Data format:* binary video objects up to 100 MiB, in the allowed video types. Outputs are an MP4 primary and an HLS ladder (`.m3u8` playlists, `.ts` segments).
- *Storage technology:* versioned S3 buckets for sources and outputs. Object storage is the only sensible store at this size. Versioning gives each source an immutable identity (VersionId), which the pinned-source rule depends on.
- *Security requirements and controls:*
  - Public access is blocked and the bucket policies are TLS-only, with SSE-S3 encryption.

    - Uploads use a presigned POST whose policy fixes the key, the content type and a length range.
    - Downloads use presigned GET links that expire after 900 s, issued only after an owner check.
    - HLS segments are served by a 307 redirect to a fresh signature.
    - CORS allows only the frontend origin.
    - The worker's read is limited to the source bucket, and the owner and source binding is checked in code before any read.
    - On AWS: tampered and expired URLs were refused, and an unsigned read returned 403 **[AWS]**.
- *Trade-off considered:* Proxying bytes through the API would centralise control, but it breaks Gateway and Lambda payload limits and costs compute. Presigned URLs are bearer capabilities for their lifetime, so the lifetime is kept short.

Asset and upload metadata (Internal; original filenames are Confidential)

- *Data format:* `assets`, `upload_reservations`, `asset_source_versions`, `request_keys`, `cleanup_intents`, `staged_objects` and `processing_projections` rows. JSONB holds operations and outputs.
- *Storage technology:* PostgreSQL in `media_db`. Check constraints enforce the pinned-source rule (`ck_upload_reservations_finalized_has_pin`) and the status sets.
- *Security requirements and controls:* Every query is filtered by owner, and a foreign ID returns HTTP 404. The runtime role cannot run DDL. Idempotency keys make retries of finalize, edit and retry safe.
- *Trade-off considered:* Media keeps a projection instead of querying Processing (AD-21). That removes a runtime dependency, at the cost of eventual consistency.

Processing job data (Internal)

- *Data format:* `jobs`, `attempts`, `submission_guards`, `retry_requests`, `attempt_tasks`, `job_messaging`, `cleanup_intents`, `launcher_control` and `launch_reservations` rows. JSONB holds the source descriptor and the outputs.
- *Storage technology:* PostgreSQL in `processing_db`. Leases, row locks and `SKIP LOCKED` need a transactional store with a trustworthy clock.
- *Security requirements and controls:* Only Processing roles hold the DSN. Owner, asset, job and source must match the stored values on every command. The worker can update only its own leased attempt.
- *Trade-off considered:* DynamoDB or the RDS Data API would remove VPC attachment and connection handling. They were left to the V2 backlog because the lease and outbox logic depends on SQL transactions.

Messages in transit (Internal)

- *Data format:* JSON envelope v1 (identifiers, `state_version`, source descriptor, payload). No bytes, credentials or URLs are allowed in a payload.
- *Storage technology:* SQS Standard queues and DLQs, plus the `message_outbox` and `message_inbox` tables.
- *Security requirements and controls:* SQS-managed encryption and TLS-only policies. Resource policies deny `SendMessage` to every principal except the listed producer roles, and receive/delete to every principal except the listed consumer roles. A direct send was refused **[AWS]**.

- *Trade-off considered:* FIFO queues would give ordering and deduplication, but at lower throughput and with message-group design work. The inbox and version fences already tolerate duplicates and reordering, so Standard queues are enough. The repository does not record a formal FIFO evaluation.

Secrets and configuration (Restricted)

- *Data format:* four role DSNs and the JWT secret as SSM SecureStrings. The Aurora master password is managed by RDS in Secrets Manager.
- *Storage technology:* SSM Parameter Store (no per-secret monthly fee) and Secrets Manager for the RDS-managed master secret.
- *Security requirements and controls:* Each handler role can read only its own domain's parameters. The master secret is reachable only by the bootstrap operator. No AWS key is stored in GitHub; CD uses OIDC. gitleaks runs in CI.
- *Trade-off considered:* SSM has no automatic rotation, and rotation is out of scope. Cold starts read the parameters once.

Operational telemetry and delivery evidence (Internal)

- *Data format:* CloudWatch logs (bounded, redacted failure events), metrics, alarm state; CI and CD artifacts; sanitized JSON receipts committed under `docs/results/` and `docs/execution/aws/v1/evidence/`.
- *Storage technology:* CloudWatch Logs (7 days), GitHub Actions artifacts (7–60 days), Git for sanitized receipts, and a private, versioned S3 state bucket for CD run state.
- *Security requirements and controls:* Signed URLs, cookies, secrets and account IDs are stripped before anything is committed. The state bucket is private because it holds the smoke account's credentials.
- *Trade-off considered:* Seven-day logs keep cost and exposure low. Incident reconstruction after a week relies on the committed receipts.

Static frontend (Public)

- *Data format:* HTML, CSS and JavaScript, with content-hashed asset names.
- *Storage technology:* private S3 bucket behind CloudFront origin access control.
- *Security requirements and controls:* Integrity comes from the deploy pipeline. The bucket is not directly public.
- *Trade-off considered:* none material.

3.2.4 Detailed Design

Three architecturally representative use cases are described here. Section 7.1 traces every use case to its classes, modules, sequence views and tests. Section 7.2 holds the shared analysis and design class models and the grouped sequence views, and section 7.3 holds the three design problems with before-and-after views.

Case 1: upload, process and play (UC-1 to UC-3). This is the platform's main flow (Figures 5–7).

Step	Class / function	What it does	Invariant protected
Reserve	`media.services.direct_intake.reserve`, `storage.direct_upload_grant`	Creates an `UploadReservation` (owner, request ID, filename, type, size, SHA-256) and a presigned POST that fixes the key, type and length; expires in 15 min	Idempotent per `(owner_id, request_id)`; changed body → 409; type and 100 MiB size checked
Upload	Browser or Publisher → S3	Sends the bytes straight to the versioned source bucket; no cookie and no API involved	Bytes never pass through Gateway or Lambda
Finalize	`direct_intake.finalize`, `storage.verify_direct_upload`, `media.messaging.enqueue_submit`	Reads size, SHA-256 and signature at an explicit VersionId, outside SQL locks; then, in one transaction, pins the version, creates the `Asset` and stages `submit`	The verified bytes are the bytes encoded; adoption and command are atomic; a missing object → 409, not adopted
Publish	`MediaRepair` → `delivery.sql.publish_due`	Claims due outbox rows (`SKIP LOCKED`), sends to `processing-commands`, acknowledges by claim token	At least once; no committed message is lost
Accept	`processing.handlers.command_handler` → `messaging.handle_command`	Records the inbox row; locks `SubmissionGuard`; creates `Job` and attempt 1 as `queued`; `emit_state` stages `work` and `result` v1	One job per asset; duplicate submit is a no-op; owner and source must match
Launch	`processing.launcher` (V2-live)	Reserves a slot in SQL and starts a Fargate task with a stable RunTask token; at most 3	The cap holds under concurrent wakes and lost RunTask replies
Claim	`processing.worker.sqs.decode_work` → `messaging.claim` → `lifecycle.claim`	Binds the task to the attempt (`AttemptTask`), takes a lease on the DB clock, sets `processing`, stages `result` v2	A valid lease is needed; one task, one attempt
Encode	`processing.worker.tasks.pipeline`, `media_engine`	Downloads the source at its pinned VersionId and checks SHA-256; ffprobe; MP4; HLS variants via `variants_for` (never above the source height)	HLS failure keeps the MP4; outputs scoped to `jobs/{job}/attempts/{attempt}/`
Finish	`lifecycle.finish`	Under the live lease, records exactly one primary output plus optional HLS, sets `done`, stages `result` v3; the worker then deletes its SQS message	Terminal state commits before the work is acknowledged
Project	`media.handlers.result_handler` → `projections.apply_result`	Inbox dedupe; `validate_transition`; upserts the projection	Never moves backwards; checks job, source and attempt namespace
Play	`projections.public_job`, `storage.sign_output`, `media/api/v1_hls.py`	Returns owner-filtered status, a presigned MP4 URL and the HLS playlist; segments redirect (307) to presigned S3 URLs	Owner only; no-store caching; short-lived capability

Case 2: retry over the stored source (UC-4).

1. `media.services.v1_lifecycle.retry` checks the owner, the projection state `failed`, the pinned source and an idempotency key (`RequestKey`).
2. It then stages a `retry` command in the same transaction.
3. In Processing, `messaging._retry_locked` locks the guard and job, and creates a **new** `Attempt` with the next number and a `RetryRequest` keyed by `(job_id, request_id)`. A replayed retry returns the same attempt.
4. If a retry arrives before its submit (SQS reordering), it is rolled back, including its inbox row, so redelivery can succeed later.
5. A worker still holding the old attempt's lease cannot finish: `lifecycle.finish` requires the current attempt and the same live `lease_token`.

No bytes are uploaded again. An expired lease marks the attempt `abandoned` and the job `failed`. The job is never re-queued automatically. A retry is always an explicit owner command, which keeps duplicate outputs and hidden loops out of the design. Tests: `test_retry_idempotency_and_cancel_fence_old_and_current_attempts` and `test_publisher_retry_reuses_the_stored_source_after_a_transient_encode_failure`.

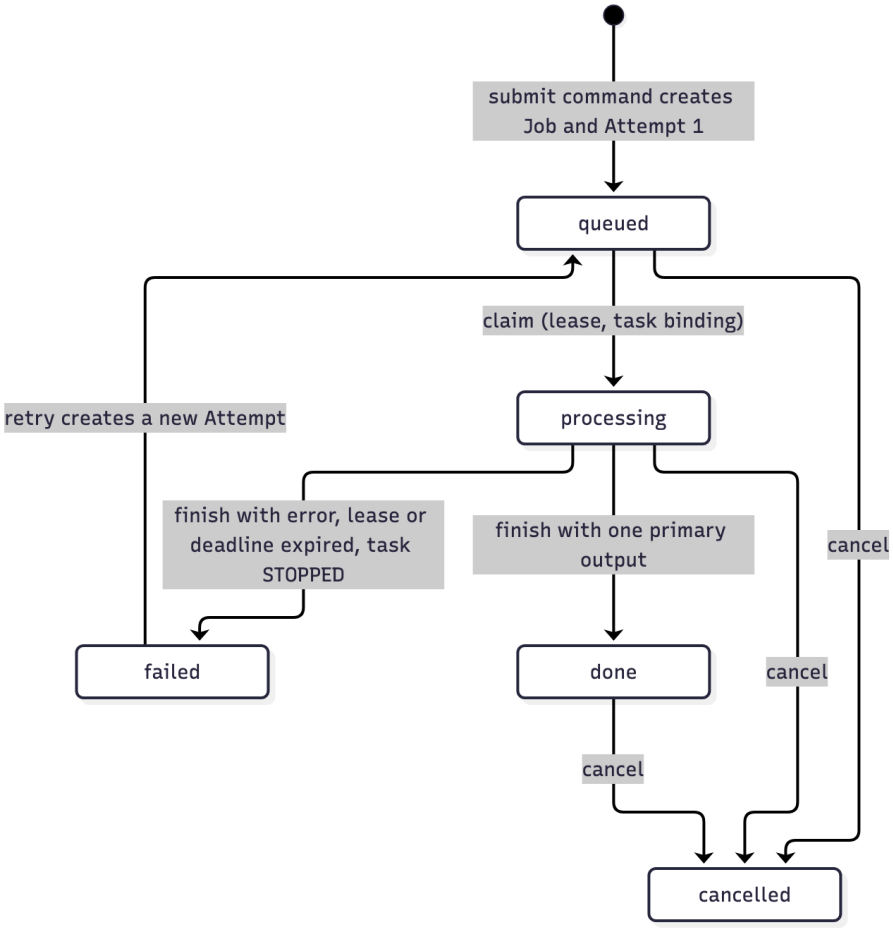


Figure 12. Job and attempt state machine: one-way transitions per attempt; a retry creates a new attempt.

Case 3: independent, safe release of one owner (UC-7).

1. `scripts/ci/cd.sh` calls `release_owner` for each owner whose image build inputs changed. Media is untouched by a Processing-only change.
2. `scripts/aws/v1/release.py` takes the journal lock and writes a durable `pending_release` intent **before** any mutation.
3. It pins the candidate image digest and the resolved task definitions. It verifies the registry manifest bytes against the scanned image config, and checks migration compatibility (heads must be unchanged, or attested as additive).
4. It publishes a Lambda version per function, moves each `live` alias, and runs the owner's smoke test.
5. On a failed gate or smoke, it restores the previous versions and records `restored`. A failed restore leaves the recovery intent in place for an explicit resume.
6. `deploy.py:adopt_release` then writes the promoted images back into the stack, so the template still describes what runs.

Schema is never rolled back. This is why migrations must be additive. Tests: `test_owner_release_applies_a_real_migration_and_failed_candidate_restores_aliases`,

`test_pending_intent_before_mutation_and_real_bad_candidate_restore` and `test_owners_are_released_only_when_their_images_build_inputs_changed`. **[AWS]**: CD run 39271754470 shipped a deliberately failing Processing candidate, and it restored automatically.

3.3 Other Architectural Decisions

The remaining register decisions, and the project decisions that shaped the architecture, are listed below. Where another section explains the rationale, the table points there.

Identifier	Decision	Rationale and trade-off	Status
AD-03	Synchronous submit with asynchronous Redis/RQ processing (proposal)	Replaced so that no always-on broker runs and Media does not depend on Processing at request time; see AD-11	Superseded by AD-11
AD-04	Idempotent `SubmitJob` keyed on `asset_id`, plus reconciliation; outbox only if needed (proposal)	Idempotency (`SubmissionGuard`, unique `asset_id`) and reconciliation (`repair_work`, lease expiry) were kept. The outbox was adopted from the start because SQL plus SQS is not atomic.	Changed (AD-11)
AD-05	Presigned URLs for source reads and output delivery (proposal)	Uploads and playback stay presigned. The worker reads the source with an IAM grant pinned to a VersionId instead, which removes a live Media callback from the encode path. This is broader than a signed URL, and owner binding is checked in code (section 3.2.3).	Changed (AD-14, AD-20)
AD-06	Service token plus forwarded owner identity on a private network (proposal)	No private network exists between Lambdas, so IAM and the Gateway-trusted actor became the boundary	Superseded by AD-09, AD-20
AD-07	Containers on AWS in one small environment (proposal)	Same intent, managed serverless mechanism. Only the encoder, migrations and bootstrap run as containers on Fargate.	Changed (AD-10, AD-12, AD-13, AD-15, AD-23)
AD-08	Fresh demonstration data; no Project 1 data migration	Keeps effort on the contribution and sets the data lifecycle (section 3.2.3). The deploy pipeline seeds one smoke account.	Kept
AD-19	Closed-by-default stack operated as booked sessions (V1)	Kept idle spend near zero, but made the platform unavailable outside sessions and cost a full deploy and teardown per check	Superseded by AD-27
AD-22	Opt-in V2 experiments in the same template, off by default: unattended launcher (#68), short encodes on Lambda (#93), one cluster per database (#94)	Keeps experiments revertible: with default flags the template is byte-identical to V1, which a test checks. #68 ran on AWS and became the V2-live default (AD-27); #93 and #94 are [NOT_RUN].	New
AD-28	**Release admission scans both the candidate and the retained rollback image**, and handles a newly published advisory that affects only the retained image without bypassing the gate (section 5.3)	A restore target must not silently reintroduce a fixable HIGH/CRITICAL finding, but a newly disclosed advisory against the *old* image must not block the release that fixes it. The policy and its evidence are in `docs/architecture/decisions.md`.	New (21 October)

Project decisions with architectural effect.

ID	Decision	Effect
F-01	Lecturer accepted the proposal and asked for detailed DDD to confirm the service choice	The DDD model and context map (section 3.1.4); AD-01 confirmed
D-01	Baseline is the team's local frozen FlickPond checkout `8854537` (the proposal named `890c5ec`)	Inherited capability is identified separately from Team 17's work
D-06	One progress report, due 16 October	Project-conduct reporting (section 2)
Freeze, 20 Oct	Team-accepted demo release frozen at `daee44b` (`docs/release/demo-release-manifest.md`); afterwards, only blocker repairs with verification and a manifest update	Later engineering releases (`629119c`, and the 21 October scan-policy fix) are recorded as post-freeze addenda with their own receipts
#271	A 503 instead of a 4xx when S3 refused an oversized upload was accepted as a known limitation at the freeze	Fixed in the later engineering release: a missing upload object now returns a recoverable 409, and real storage failures remain 503. The 100 MiB cap was enforced throughout.

Rationale already given elsewhere is not repeated here:

- Performance decisions (caching, scale-to-zero, direct-to-S3): section 4.1.
- Availability (outbox, leases, repair, DLQs): section 4.2.
- Security (IAM, authorizer, scanning): section 4.3.
- Maintainability (contracts, per-service lanes, legacy quarantine): section 4.4.

3.4 Architectural Limitation

The key outstanding architectural issues are listed below. Owners follow the agreed specialisms (cloud and delivery: Guruprasath Gopal; backend services: Ibrahim Mammadov; domain, contracts and verification: Nguyễn Kim Long).

The **timeline** has two horizons:

- **C**: within the course, by 16 November 2026.
- **Post-course roadmap**: applies if FlickPond is taken forward. Phase **R1** is the first month, **R2** is months 2–3 and **R3** is later.

The persistent environment is torn down after 16 November 2026. Any post-course phase would start from the repository and the retained template.

Identifier	Issue and impact	Description	Resolution	Owner	Status
AISS-01	Shared database failure and capacity boundary	One Aurora cluster with one instance (0–2 ACU) holds both logical databases. An outage, a failed resume or the capacity ceiling affects both services. Backups are kept 1 day, and no restore drill was run.	R2: deploy the already-rendered one-cluster-per-domain option (#94) after an AWS run; add a reader in the second AZ; run a quarterly restore drill	Guruprasath	Accepted for course scope
AISS-02	No regional or zone failover	Everything runs in ap-southeast-1. The subnets span two AZs, but the database instance does not. Multi-zone and multi-region failover were excluded in the proposal.	R3: Multi-AZ Aurora reader, cross-region backup copy, documented RTO/RPO	Guruprasath	Accepted (out of scope)
AISS-03	Queue wait misses its target: about 37 s against 20 s	Fargate task start (median 25.6 s, n = 12) dominates the wait from `queued` to `processing` for a warm job **[AWS]**	R1: measure a slimmer worker image and lazy image loading; R2: run the opt-in Lambda short-encode lane (#93) for short clips	Long (verify), Guruprasath	Open, measured miss
AISS-04	Two-worker scaling miss and an evidence gap	AWS V1 measured 1.449× against 1.6× at two workers (2.347× against 2.1× at three passed), from a 6-clip backlog with one run per cap. The proposal's 30-clip AWS comparison was not run.	R1: rerun 30 clips × 3 repeats per cap with per-task CPU profiling; report against the unchanged targets	Long	Open, measured miss
AISS-05	Bounded capacity, no general autoscaling	At most three concurrent encodes (a cost choice; the Fargate quota is now 30 vCPU). Work beyond the cap waits in the queue.	R2: raise the cap behind the budget alarm; size the cap from queue age	Guruprasath	Accepted (cost)
AISS-06	Public API endpoint beyond CloudFront, no WAF	The `execute-api` URL is reachable directly. Only the authorizer and a single stage throttle protect it.	R1: CloudFront-only origin (secret origin header checked by the authorizer) plus AWS WAF managed rules	Guruprasath	Open
AISS-07	Owner-only access	Only the owner can play an asset. The contract accepts a `viewer` role that nothing issues, so there is no sharing.	R3: share grants in Media, issuing `viewer`; no contract version change is needed	Ibrahim	Accepted (out of scope)
AISS-08	Image-only rollback	Release restores images and aliases, never schema. Pair compatibility is an operator attestation backed by N/N-1 contract tests.	R2: enforce expand-and-contract migrations with an automated old-image-on-new-schema test in CI	Ibrahim	Accepted, mitigated by policy
AISS-09	Broad deploy role; one environment	The CD role has `AdministratorAccess` with a region lock. Promotion to the single environment is automatic after green Main CI; there is no staging and no reviewer gate on the `aws` environment.	R1: least-privilege deploy policy; required reviewers on the `aws` environment; R2: a staging stack from the same template	Guruprasath	Open

Identifier	Issue and impact	Description	Resolution	Owner	Status
AISS-10	Legacy residue	Peer-era tables and columns (`retry_intents`, `jobs.source_url`) and the `legacy/` package remain. They are kept out of images and unused.	R2: a separately planned contract migration once the last legacy stack is gone	Ibrahim	Accepted, scheduled
AISS-11	Alarms with nobody paged	13 alarms and a dashboard exist, but there is no notification target, no tracing, and logs last 7 days. No alarm was observed firing.	R1: SNS notification target, an alarm drill and AWS X-Ray	Guruprasath	Open
AISS-12	Undefined production data retention	There is no account deletion, no purge of tombstones or inbox/outbox rows, and no S3 lifecycle rule. For this deployment, all demonstration data is removed at the teardown after 16 November, and the final Aurora snapshot is deleted once grades are released.	C: teardown after 16 Nov 2026; R2: define production retention (noncurrent-version expiry, 30-day inbox/outbox pruning, tombstone purge)	Long	Open (production); planned (demo)
AISS-13	Base-image vulnerabilities	Images carry unfixed (no-fix-available) findings: 44/44/208 rows at the last scan. A new advisory against a retained rollback image blocked CD on 21 October (section 5.3).	Closed for the release path by AD-28; R1: a scheduled weekly rebuild and rescan of retained images; R2: a smaller worker base image	Guruprasath	Closed (gate); Open (hardening)
AISS-14	Evidence not established	Not established: the AWS lost-queue-entry timing (120 s bound), the duplicate-output check after a worker kill, a restore drill, the Publisher's 2 person-day effort, and actual spend (Cost Explorer showed USD 0.00 with credits applied).	C: read the final October bill in Cost Explorer before 16 Nov; R1: run the remaining experiments with retained receipts	Long, Guruprasath	Open
AISS-15	At-least-once delivery	Duplicates and reordering are expected. They are absorbed by inbox and version fences: 18 sends were recorded for 9 jobs. Poison messages stop in a DLQ and need an operator redrive (runbook).	None needed; this is by design and documented	Ibrahim	Accepted

4. Quality Attributes

Each attribute below gives the design tactics first, then the evidence against the proposal's targets (proposal §3.4, unchanged). Results keep their environment, source, workload and timing boundary. Local results never stand in for AWS results, and misses stay misses. The complete record is in `docs/results/actual-vs-target.md` (with its CSV) and `docs/evidence-index.md`.

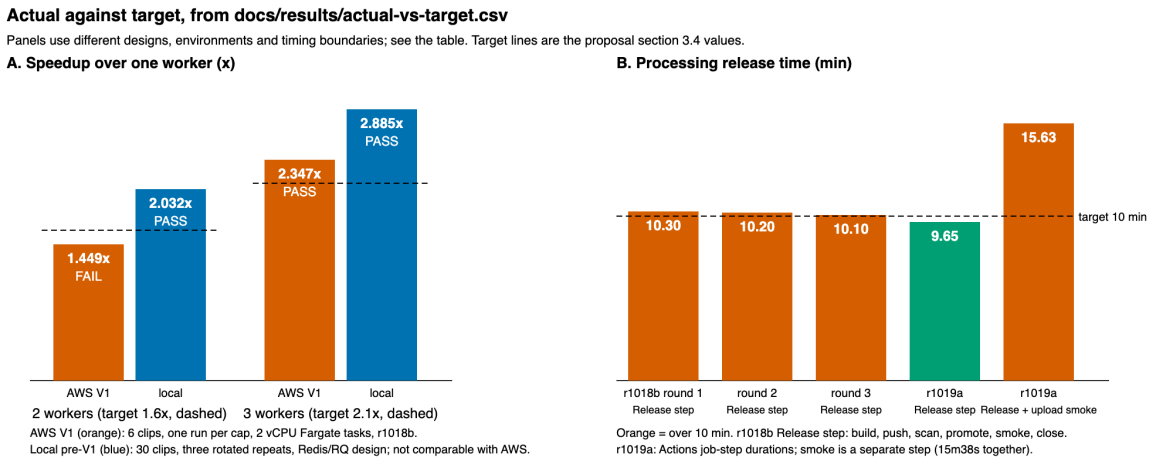


Figure 13. Speedup and Processing release time against target (AWS V1 and V2-live; misses shown as misses).

Attribute	Scenario (proposal target)	Result	Verdict
Scalability	2 workers ≥ 1.6× and 3 workers ≥ 2.1× the jobs/min of 1 worker	AWS V1 (6 clips, one run per cap): 0.483 / 0.700 / 1.134 jobs/min → **1.449×** and **2.347×**	**FAIL** at 2; PASS at 3
Scalability	Median queue wait falls at each step; 0 failed jobs	274.3 s → 135.7 s → 9.6 s; 0 failed; 0 encoders left running	PASS
Scalability	(same, local pre-V1 design, 30 clips × 3 repeats)	2.032× and 2.885×	PASS (different design; not AWS)
Scalability	V2-live automatic scale 0 → 3 → 0	0 → 3 → 0 with 3 concurrent clips; 9-clip backlog at cap 3 in 239 s (2.26 jobs/min)	PASS (no 1/2/3 comparison)
Performance	50 users polling status for 3 min: p90 ≤ 700 ms, < 1 % errors	p90 **58.2 ms**, 9,000 requests, 0 errors (in-region generator)	PASS
Performance	20 concurrent logins: p90 < 2 s	p90 **178 ms** (20 samples)	PASS
Performance	Warm-job queue wait ≤ 20 s (V2-live)	about **37 s**; Fargate start median 25.6 s	**FAIL**
Availability	Worker killed mid-encode: re-queued or failed within 5 min	Stop detected in **5.8 s**; lease expiry marks the job `failed`; retry creates a new attempt	PASS (reaction time only)
Availability	Lost queue entry re-enqueued within 2 min	23.5 s (local pre-V1); ≤ 120 s asserted on a simulated clock (local V1)	PASS locally; **NOT_RUN** on AWS
Availability	Processing temporarily unavailable while Media accepts uploads	Command consumer off for 48 s: Media answered 202 and held the command; job `done` 58.9 s after re-enable	PASS (consumer only)
Deployability	Release Processing alone, smoke-tested, in ≤ 10 min; Media unchanged	Operator promote 322.5 s; CD `Release` step 10.1–10.3 min (`r1018b`) and 9 m 39 s (`r1019a`); release plus a separate upload smoke 15 m 38 s; Media unchanged at every snapshot	**FAIL** (marginal); not established with the smoke
Deployability	Bad candidate restores automatically	CD run 39271754470 (`bad_candidate=true`) → `restored`	PASS (source `7928b7c`)

Attribute	Scenario (proposal target)	Result	Verdict
Security	Negative tests 100 % rejected; 0 fixable HIGH/CRITICAL at release	All selected negatives rejected on AWS V1 and on `dc2-live` (#229); 0 fixable HIGH/CRITICAL at each release	PASS (checks run)
Reusability	Publisher CLI: 0 service imports, 0 database credentials, ≤ 2 person-days	0 and 0, enforced by a test; 491 lines; effort not timed	PASS; effort **NOT_MEASURED**

4.1 Performance

Tactics in the design.

- **No bytes through the API.** Uploads go straight to S3 by presigned POST. Playback reads MP4 (with range requests) and HLS segments straight from S3 by presigned URL. Lambda handles only small JSON requests, and there is no payload limit to hit.
- **Caching where it is safe.** CloudFront caches the content-hashed frontend at the edge. API responses deliberately use the AWS managed *caching-disabled* policy, and authenticated responses carry `Cache-Control: private, no-store`: status and signed URLs are per-owner and short-lived, so caching them would leak or go stale. HLS playlists are never cached.
- **Load balancing is managed.** API Gateway and Lambda spread requests across concurrent executions. SQS distributes work across up to three encoders, and each encoder pulls one attempt at a time.
- **Reads come from a local projection.** Status is answered from Media's own projection in one owner-local query (an earlier N+1 lookup was replaced by one outer join). There is no cross-service call per poll.
- **Bounded, predictable encoding.** Each encoder is a 2 vCPU / 4 GiB task. FFmpeg uses every core it sees, so the per-task cap is what makes the 1/2/3 comparison fair. The HLS ladder never exceeds the source height, so no work is wasted upscaling.
- **Connections suited to a pausing database.** Per-invocation sessions (`NullPool`) let Aurora pause at 0 ACU. The first read after a pause costs about 4.7 s; later reads take about 263 ms **[AWS]**.

Results and explanation.

- **Latency.** Status p90 was 58.2 ms against 700 ms, and login p90 was 178 ms against 2 s **[AWS]**. The generator was a Fargate task in the same region, so these numbers exclude internet latency to end users.
- **Two-worker miss.** The 1.449× result is measured on six clips with one run per cap. Fargate task-time variability against a fast single-worker baseline is the most probable cause, but no per-task CPU profile was captured, so the cause is an inference. The three-worker result (2.347×) passed, and the median queue wait fell at each step.
- **Queue-wait miss.** The V2-live wait of about 37 s is a deliberate trade-off of scale-to-zero: no encoder runs idle, so each burst pays a Fargate start (median 25.6 s) plus image pull. The target was not revised, and AISS-03 records the options.

4.2 Availability

Tactics in the design.

- **Durable state first.** SQL is the source of truth, and the outbox commits every message with its state change. A lost queue therefore delays work but cannot lose it: `repair_work` rebuilds queued work from `jobs` and `attempts` on the one-minute repair tick. ECS `STOPPED` and S3 `Object Created` events trigger the same pass early.
- **Failure isolation between services.** Media never calls Processing. While Processing's command consumer was off, Media kept accepting uploads (202), and the job completed after re-enable **[AWS]**.
- **Fenced, disposable workers.** Leases on the database clock, task binding and a hard deadline mean a dead or stuck worker cannot finish late. Its attempt is visibly `abandoned`/`failed`, and the owner retries from the stored source. The worker cannot send messages, so a dying worker cannot publish a half-finished result.
- **Poison-message containment.** Each queue redrives to a DLQ after five receives. A poison message reached the DLQ after 2,101 s and was removed **[AWS]**. A DLQ runbook covers redrive.
- **Managed redundancy where it is free.** Lambda, SQS, S3, API Gateway and CloudFront are regional or global managed services spanning multiple AZs. The subnets cover two AZs.
- **Self-healing deploys.** A failed release smoke restores the previous images automatically (section 5.3). Stateful resources use `DeletionPolicy: Retain`, and CloudFormation refuses a change set that would replace Aurora, a bucket or the distribution.
- **Observability.** Per-service alarms (DLQ depth per queue, errors per function: 13 in V2-live) and one dashboard. 13 of 13 alarms were `OK` after the backlog tests **[AWS]**.

Limits, stated plainly.

- The platform is **not highly available** in the strict sense. One Aurora instance in one region is a shared failure boundary (AISS-01, AISS-02).
- Backups are kept one day, and no restore drill was run.
- No uptime SLA is claimed.
- The lost-queue-entry timing was not measured on AWS (AISS-14).

4.3 Security

Controls in the solution.

Area	Control
Identity and access	Media-owned Lambda authorizer on every route except health, register, login and logout. It verifies the HS256 session JWT and re-reads the current user and role. The Gateway adapter strips caller-supplied `x-owner-id`, `x-role` and cookie headers and builds the actor only from the authorizer context. Owner filters apply in both services, and a foreign ID returns 404. Admin routes need the `operator` role.
Session	Host-only, `HttpOnly`, `Secure`, `SameSite=Strict` cookie through CloudFront; 1-hour token; `private, no-store` on authenticated responses
Hashing	bcrypt (cost 10) with a password byte limit; SHA-256 over every uploaded source and every output
Encryption	TLS everywhere: CloudFront HTTPS only (TLS 1.2+ to the origin), TLS-only S3 and SQS policies, `rds.force_ssl=1` with verified CA. At rest: Aurora storage encryption, SSE-S3, SQS-managed encryption, SecureString parameters.

Area	Control
IAM (least privilege)	One role per Lambda, migration task and worker (17 roles at default flags). Queue resource policies deny unlisted producers and consumers. Separate database roles; native role credentials; no static keys anywhere. GitHub reaches AWS only through OIDC.
Input validation	Type allow-list and a 100 MiB cap, enforced at four points: the frontend, the grant, the S3 POST policy and the Publisher. Finalize checks size, hash and file signature at a pinned version. Envelope validation rejects secret- or URL-like payloads. HLS file names are allow-listed.
Network	No inbound security-group rules except the database's internal rule; no NAT; private subnets for Lambdas and Aurora; only ports 80/443 public (80 redirects)
Secrets	SSM SecureStrings scoped per role; RDS-managed master secret; gitleaks secret scanning; receipts sanitized before commit
Supply chain and testing	Ruff security lint (flake8-bandit rules); Trivy on all three exact images in CI and again on the deployed builds in CD (zero *fixable* HIGH/CRITICAL gate); ECR scan on push; authenticated OWASP ZAP DAST; dependency audits (npm audit: 0 findings)
Intrusion detection and prevention	Not deployed. There is no WAF, GuardDuty or IDS/IPS (AISS-06). The compensating controls are the authorizer, the stage throttle (20 rps, burst 50), the budget alarm and the per-service error alarms.

Verification. The proposal's negative tests were run on AWS: on V1 stacks `r1017b`/`r1018b`, and in full on the persistent `dc2-live` environment for issue #229 on 19 October **[AWS]**.

Control (proposal §3.3)	Result
Unauthenticated requests rejected	401 on all six probed job routes, through CloudFront and directly through Gateway
User A cannot read, retry, edit, delete or play User B's assets	404 for B on every Media and Processing route for A's job
Forged identity headers and token-less internal calls	Forged headers changed nothing; anonymous admin 401, non-operator 403
Direct queue send by an unlisted principal	403 on all three queues; 0 messages accepted
Each service's credentials refused on the other's database	`InsufficientPrivilegeError`; runtime DDL denied
Expired and tampered presigned URLs; no public listing	Tampered 400/403; both 900 s expiries waited out (206/204 before, 403 after); unsigned S3 403
Invalid and oversized uploads	100 MiB + 1 byte refused by S3 (400) and never adopted
External port scan; private datastores	Only 80 and 443 open; Aurora `PubliclyAccessible=false`; no `0.0.0.0/0` ingress
Secrets out of git and images	gitleaks green in CI
Zero fixable HIGH/CRITICAL image findings at release	Met at every release, with the retained-image incident described in section 5.3

Scope limits.

- "Zero fixable HIGH/CRITICAL" never means zero vulnerabilities. The images still carry unfixed findings (44/44/208 rows at the last scan), and an earlier accepted set included one CRITICAL with no fix available.
- DAST findings are report-only, cover seven selected routes, and ran against the local gateway, not the AWS edge. Eight low and eight informational entries remain, mostly missing hardening headers on the local bridge.
- SonarCloud analysis is advisory and was not configured with a token.
- The #229 negatives ran before the final frozen source and were not repeated on it.

4.4 Extensibility and Maintainability

How the platform is extended.

- **A new client application.** It uses only the two OpenAPI 1.0.0 documents and the public flow: reserve, upload, finalize, status, retry. The Publisher CLI is the worked example: 491 lines of Python in 6 files, using only the standard library and `httpx`. It has 0 imports from service code and 0 database credentials, enforced by an AST test, and it is validated against the published OpenAPI schema in CI.
- **A new processing capability.** Add an operation to `media_engine/` (pure command builders) and to the operation list carried in `submit`. Additive payload fields need no contract version change, because consumers ignore unknown fields. The edit feature (clip, crop, scale, convert) was added this way without touching the transport.
- **A contract change.** The policy in `contracts/README.md` applies. Changes are additive within a version. Consumers accept versions N and N-1, and producers emit the lower version until every consumer reads the new one. Each consumer owns a fixture of the fields it reads, and the producer's tests check its real messages against that fixture. A breaking change takes two releases, in either service order.
- **More capacity.** Raise the encoder cap behind the budget alarm (AISS-05). Media is unaffected.

What keeps it maintainable.

- **Boundaries are enforced by tests and builds.**
  - Image builds fail if the opposite domain or a legacy module is importable.
  - IAM permissions are generated from a code-to-permission map and asserted in `tests/aws/v1/test_infrastructure.py`.
  - The checked-in template is byte-compared with its generator.
- **Per-service CI lanes.** Path filters select tests and builds per service. `CODEOWNERS` maps the directories to the specialism leads.
- **Tests and coverage.** The complete offline regression is 3,697 tests (plus 177 frontend tests) **[Local]**. The five backend coverage tiers pass their floors (Media 93.91 % combined, Processing 93.90 %). Frontend coverage is 96.24 % of statements against a 90 % floor.
- **Recent maintainability work.**
  - Shared storage-credential policy: 253 → 175 lines.
  - Owner-image dependency exports: 44 → 40/36/33 packages.
  - The frontend dependency tree went from 214 to 171 packages, with 9 npm audit findings removed.
  - An obsolete 300-line Compose file and unused CI services were deleted.
  - Ruff formatting was applied everywhere except released migrations, whose bytes are protected by a release guard.
- **Retired code is quarantined, not deleted.** The `legacy/` package keeps the Project 1 and proposal-era paths testable without shipping them (AD-26).
- **Documentation sits next to the code.** The ADR register, DDD model, views, use-case catalogue, runbooks and sanitized receipts are all in the repository. A study guide is published from the same sources (GitHub Pages).

Limits.

- `contracts/` is still a shared kernel copied into both images.
- There is one lockfile and one CloudFormation stack for both services (AD-23).
- Only contract version 1 exists, so the N-1 path is exercised by tests rather than by a real second version.
- The Publisher's two person-day build-effort target was not measured. Its first commit was on 13 October, with changes on 15 and 18 October.

5. DevOps and Development Lifecycle

5.1 Source Control Strategy

Repository strategy: one monorepo. Everything lives in one public GitHub repository, `Collaboration95/dc2`, with default branch `Main`: both services, both clients, the shared kernel, infrastructure as code, pipelines, tests, evidence and documentation. One repository keeps a contract change and both of its consumers in a single pull request, which the contract policy requires. Service independence is enforced inside the repository by path-filtered CI lanes, per-service images and CODEOWNERS, not by separate repositories.

Path	Contents	Release unit
`media/`	Media Management service, its Alembic migrations	Media image
`processing/`, `media_engine/`	Processing service, finite worker, FFmpeg command builders	Processing and worker images
`contracts/`, `delivery/`, `runtime_support/`	Shared kernel: published language, outbox/inbox and SQS adapters, Lambda runtime	Copied into both images; a change releases both
`frontend/`, `publisher/`	The two clients	Frontend bucket; Publisher package
`deploy/`, `scripts/aws/`	Image recipes, CloudFormation generator and template, deploy, session, release and evidence CLIs	Stack and operator tooling
`.github/workflows/`, `scripts/ci/`	CI, CD, DAST, V1 verification and Pages workflows; CI policy and CD scripts	Pipeline
`tests/`	Unit, integration, contract, AWS-offline, CI-policy and browser tests (3,982 Python tests)	n/a
`docs/`, `project-documentation/`	Architecture, ADRs, runbooks, sanitized receipts, course documents, this report	n/a
`app/`, `legacy/`, `migrations/`	Inherited Project 1 application and retired peer-HTTP/Redis code, kept testable but excluded from images	Not deployed

Branching strategy: trunk plus short-lived integration branches.

- `Main` is the trunk. A push to `Main` triggers full CI, and a green run deploys automatically.
- Feature and fix branches (`fix/*`, `docs/*`, `v1/*`, `v2/*`, `ci/*`, `release/*`) are short-lived. They merge by pull request into an **integration branch**: `dev/v1-integration` then `dev/v1` for the V1 build (16–17 October), and `dev/closeout` for close-out (17–21 October).
- Each integration branch merges into `Main` through one release-style pull request that runs the full verification.
- Merge commits are used throughout, so each PR's history stays visible on `Main` (140 merge commits).
- Every PR body records its "Tech decisions".

- The only tag is `project2-v1`, which marks the V1 release (18 October). Later releases are identified by source SHA and image digest in sanitized receipts.

Era	Branch flow	Example PRs
Local waves (12–14 Oct)	`wave/*`, `fix/*` → `Main`	#28–#30 (Processing, Media, clients)
AWS preparation (14–15 Oct)	`work/aws2-*` → `work/aws2-prep` → `Main`	#70
V1 build (16–17 Oct)	`v1/*` → `dev/v1-integration` → `dev/v1` → `Main`	#112, #136, #194
Close-out (17–21 Oct)	`fix/*`, `v2/*`, `docs/*` → `dev/closeout` → `Main`	#239, #264, #269, #274, #277
Quality and hand-off (20–21 Oct)	`quality/*`, `docs/*` → `Main`	#278–#284

Pull-request checks. CI runs on every pull request:

- Into `dev/closeout`, it runs only the lanes whose inputs changed. The aggregate check is **Dev gate**.
- Into `Main`, it runs every lane. The aggregate check is **Full verification**.

Since 15 October, every first-parent commit on `Main` is a PR merge, and 26 of the 27 PRs into `Main` merged with all checks green. The exception, #194, merged with the advisory third-party GitGuardian check failing.

Authentication and access of developers.

- The repository is owned by one personal GitHub account (`Collaboration95`), which is its only collaborator. All commits, PRs and issues were made through it, by the team and by the AI coding agents it ran.
- **Branch protection and rulesets are not configured.** “Dev gate” and “Full verification” are stable, enforceable check names, but nothing in the repository settings requires them, and no reviewer approval is required. Pull-request discipline was a team practice, not a setting.
- Secret scanning with push protection is enabled, and gitleaks runs in CI.
- Cloud access is separate from repository access:
    - Human operators use IAM users with an MFA session helper (`make aws-mfa-session`).
    - The automation agent uses a dedicated IAM user (`flickpond-agent`, profile `dc2-agent`).
    - GitHub Actions uses **OIDC only**. No AWS key is stored in GitHub. The deploy role trusts only the immutable subject of this repository’s `aws` environment, and that environment admits only `Main` and `dev/closeout`.

The missing protection is recorded as AISS-09. For a multi-account team, the next steps are:

- add the members as collaborators;
- require “Full verification” and one review on `Main` and `dev/closeout` through a ruleset;
- require reviewers on the `aws` environment.

5.2 Continuous Integration

`.github/workflows/ci.yml` is the integration pipeline. Its reusable parts are `v1.yml` (V1 local verification) and `dast.yml`.

Triggers.

Trigger	Mode	Purpose
Pull request into `dev/closeout` or another integration branch	dev (selected)	Fast feedback: only lanes whose path inputs changed; aggregate check **Dev gate**
Pull request into `Main`	full	Every lane before integration; aggregate check **Full verification**
Push to `Main`	full	Verifies the exact merged source; a green run is CD's admission ticket
Manual dispatch (`candidate_sha`)	full	Verify an exact candidate
Weekly schedule (Sunday 03:00 UTC)	full	Catches drift: new advisories, dependency and tool changes

`scripts/ci/policy.py classify` decides the mode and the selected lanes, and writes them to a `ci-selection` artifact. It has its own offline tests. The final gate job requires every selected check to pass and validates every intentional skip, so a lane cannot silently disappear.

Jobs.

Job	What it runs
Select verification policy; CI policy offline tests; planning validation	Lane selection and its tests
Lint and Python unit tests	Locked `uv` environment; Ruff lint and security rules; format check; the fast pytest tier with coverage
Frontend tests	`npm audit --audit-level=high`; Vitest with V8 coverage (90 % floors)
Legacy real integration	PostgreSQL 16, MinIO and FFmpeg integration of the inherited app
Processing worker, recovery and database ownership	Owned PostgreSQL and storage acceptance in Docker
Media owned persistence and boundary contracts	Media migrations and integration tests on PostgreSQL
Media service; Processing service	Per-service tests, then **build the exact service image** and verify its runtime (no opposite domain, no legacy modules, handlers start)
Independent Publisher HTTPS and cookie persistence	The Publisher against real TLS
Local HTTPS and browser trust	Playwright and Chromium against the local HTTPS stack: browser, public API and Publisher acceptance
V1 local verification (`v1.yml`)	cfn-lint (required); offline control-plane tests (template, IAM, release, launcher); V1 flows on PostgreSQL, Moto and FFmpeg with a browser leg; build of all three images, then **Trivy on the exact images**, failing on any fixable HIGH/CRITICAL
Combined backend package coverage	Merges five coverage tiers; at least 80 % per package
Scan container image (app / media / processing)	Trivy image scan matrix; reports kept 60 days
Secret scan (gitleaks)	Full history with an allow-list
Exact-candidate DAST (`dast.yml`)	Authenticated OWASP ZAP against the local Media and Processing APIs; findings report-only
SonarCloud SAST (advisory)	Skipped while Sonar is unconfigured
Dev gate / Full verification	Policy gate over the selected checks

On the latest `Main` run (39531399249, source `4ae6a0b`), all 25 selected jobs passed and the 3 skips were intentional. Wall-clock time was 7 min 9 s, because the jobs run in parallel; the longest job, the browser trust suite, took 405 s. Across the project, CI ran 246 times. Artifacts are bound to their source SHA: coverage fragments, image scan reports, DAST findings and selection records.

5.3 Continuous Delivery

`.github/workflows/cd.yml` and `scripts/ci/cd.sh` automate delivery.

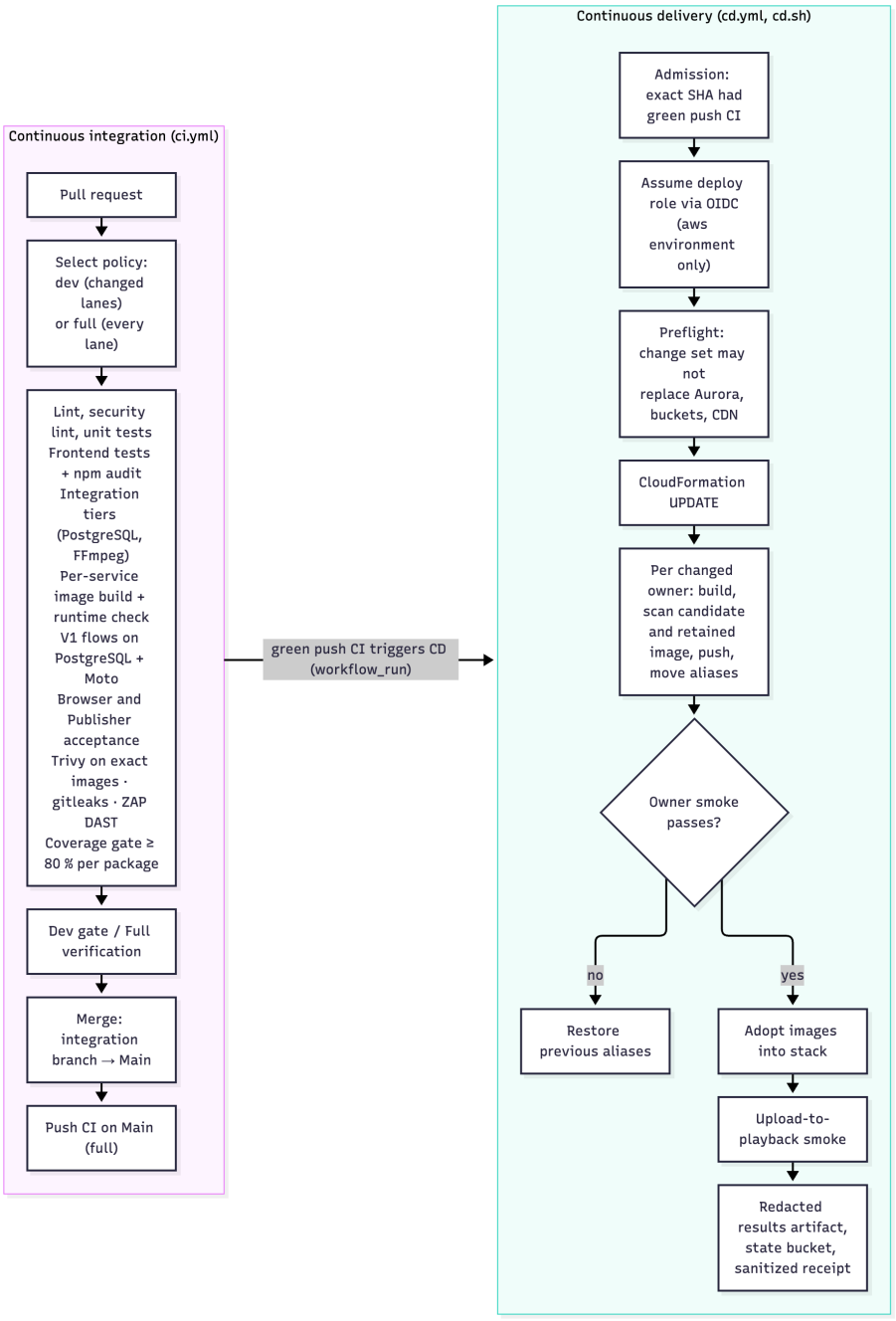


Figure 14. CI/CD pipeline: pull-request lanes, full verification on Main, exact-source admission, in-place stack update, per-owner release with scan, smoke and restore, and the receipts it retains.

Environments.

Environment	Purpose	How it is created
Developer machine	Fast checks; the full local V1 stack on PostgreSQL, Moto and FFmpeg, with the same images	`make setup-v1`, `make v1-up`, `make v1-demo`
CI (ephemeral)	Exact-source verification, image builds and scans, DAST	Every pull request and push
AWS V1 per-run stacks (historical)	The proof runs `r1017b`, `r1018b`, `r1019a`, each deleted afterwards	Dispatched deploy and teardown (AD-19)
AWS `dc2-live` (production demo)	The one persistent environment, updated in place	First deploy by a human dispatch; then automatically after every green CI run on `Main` (AD-27)

There is no separate staging environment (AISS-09). Two things stand in for it: the local V1 stack and CI's exact-image verification exercise the same images before promotion, and every release smoke-tests production with an automatic restore on failure.

Promotion and who approves it.

- Promotion **into `Main` is the approval**. The person merging the release PR is the approver; in this project that was the repository owner, Guruprasath Gopal, the cloud and delivery lead.
- After that, promotion to `dc2-live` is automatic, with no manual gate:
    1. CI on `Main` completes and triggers CD (`workflow_run`).
    2. The **admission gate** job refuses the commit unless that exact SHA had a successful push CI run in this repository, then retains an admission record.
    3. The `deploy` job assumes the AWS role through OIDC, only from the `aws` environment.
- Manual dispatch remains available for `preflight`, `deploy`, `release` (including a deliberate `bad_candidate`), and `teardown`. `teardown` requires `confirm=live`.
- A concurrency group prevents two deliveries running at once.

Deployment steps and verification.

1. **Preflight** (reads only): account and region, stack state, a change set that must not replace Aurora, a bucket or the distribution, and the release journal.
2. **Stack update**: a CloudFormation `UPDATE` change set from the checked-out template.
3. **Per-owner release**, only for owners whose image build inputs changed:
    - build the image;
    - scan the candidate, and scan the retained rollback image (AD-28);
    - push it, and bind the registry manifest to the scanned config;
    - run the migration task if attested;
    - publish versions and move the `live` aliases;
    - run the owner's health and authentication smoke;
    - **restore the previous versions** if anything fails;
    - adopt the images into the stack.
4. **Upload-to-playback smoke**: a real clip is uploaded through the public API. The job must reach `done`, and the MP4, HLS playlist and segment must be playable.
5. **Receipts**: redacted results are uploaded as an artifact. Run state is saved to a private, versioned S3 bucket. Sanitized delivery receipts are then committed (for example `docs/results/engineering-20261020/delivery-629119c.json`).

Recorded deliveries.

Run	Source	Outcome
39085599276 / 39087966647 / 39089615147 (`r1019a`)	`0b491e1`, `358d169`	V1 deploy with smoke PASS; Processing release 9 m 39 s with post-release smoke PASS; teardown `CLEAN`
39105817431	`d2070db`	First persistent `dc2-live` deploy; smoke PASS
39271754470	`7928b7c`	Deliberate bad Processing candidate: smoke failed (503) and the previous aliases were **restored** automatically
39286933644	`daee44b`	Deploy of the team-accepted frozen demo release
39416747925	`629119c`	Both owners promoted, deployed-image scans clean, upload-to-playback smoke passed, read-back of 3 images, 10 aliases and the worker definition
39532075705	`4ae6a0b`	**Failed**: the scan of the retained Media rollback image found 1 fixable HIGH/CRITICAL (see below)

The 21 October scan incident. This was the only time the release gate blocked a correct release. The handling shows how the pipeline is meant to be operated.

1. **Detection.** CD run 39532075705, for `Main` at `4ae6a0b` (which bumped `certifi` for a new advisory), failed in `Deploy` with `media: FAIL: Trivy has 1 fixable HIGH/CRITICAL`. The upload-to-playback smoke was skipped. The live environment kept running the previous verified release, `629119c`, untouched.
2. **Root cause, from the run log and the saved reports.** The candidate image passed its scan. The failing scan was the rescan of the image running at the time, the rollback baseline (`sha256:24bd78a8…`, built from `629119c`). It carried GHSA-6q48-6c7w-9gw2 in `certifi 2026.7.22`: emSign root CAs with compromised intermediate keys. The advisory was published on 20 October at 19:35 UTC, after that image shipped. The candidate's `certifi 2026.10.19` fixes it. So the gate refused to replace a vulnerable image *because* it was vulnerable, and the fix could never ship.
3. **Fix without weakening the gate (AD-28, PR #285 into `dev/closeout`, PR #286 into `Main`).** The rules are now:
    - The candidate must still scan clean first.
    - Images that the release **keeps** are rescanned strictly.
    - Images that the release **replaces** are rescanned in a *retained* mode. A fixable finding there becomes `RETAINED_FIXABLE`: it is listed in the release receipt, annotated on the run and shown in the job summary, and it does not block.
    - Only the baseline role accepts that status. Promotion, publication and the good and bad image sets still require `PASS`.
    - Scanner errors and wrong-image reports fail in both modes.
    - Tests pin each rule. The real failing report, replayed through the scanner, fails in strict mode and is recorded in retained mode.
4. **Redelivery.** {{CD-REDELIVERY}}

This incident motivates the scheduled weekly rescan of retained images (AISS-13).

6. Other Things to Be Highlighted

6.1 What Team 17 Contributed beyond the Inherited Baseline

Project 2 did not add product features to FlickPond. It changed how FlickPond is **built, deployed and reused**. The contribution, measured against the inherited baseline:

- **A domain-driven split** into two bounded contexts. Each has an aggregate with invariants enforced by constraints and code, its own database and roles, and its own image, IAM roles, CI lane and release unit (sections 3.1.3 and 3.1.4).
- **An asynchronous, failure-isolated integration**: outbox, SQS and inbox in both directions, version-fenced projections and SQL-driven repair. Media keeps working while Processing is unavailable (section 4.2).
- **A second client** built only against the published contract, proving reuse (section 4.4).
- **A cloud-native runtime that costs almost nothing idle.** No servers, no NAT, Aurora at 0 ACU, and encoders that exist only while there is work (section 3.2).
- **A delivery pipeline that releases one owner at a time from immutable, scanned images**, and restores itself on a failed smoke test (section 5).
- **An evidence discipline.** Every claim in this report links to a test, a CI run or a sanitized cloud receipt labelled with its environment and source. Misses and not-run items are kept.

6.2 Lessons Learned

1. **Design for the transport you actually have.** The proposal assumed synchronous HTTP between the services. Once both ran as Lambdas with no private network, a transactional outbox and inbox became necessary, not optional. Adopting them early (AD-11) avoided a dual-write class of bugs.
2. **"Scale to zero" moves cost into latency.** Scale-to-zero made the platform cheap to keep running, which is why V2-live replaced booked sessions (AD-27). The same choice is the main cause of the 37-second queue-wait miss. Both effects are measured and reported.
3. **Idle is a behaviour that needs testing.** Aurora never paused at first, because cleanup rescans woke it every few minutes. #273 added activity markers and backoff, and a CloudWatch reading of 0 ACU confirmed it.
4. **Pipelines need tests too.** Two CD script bugs (#265, #267) and one workflow-startup error (#281) stopped deployments partway. All were fixed with tests and `actionlint`. Formatting once changed a released migration's bytes, so Media's release was held (#282); migrations are now excluded from formatting and guarded.
5. **At-least-once delivery means duplicates in practice.** A burst produced stale duplicate work messages that broke a smoke test (#270). The fix drains them, and the re-test still recorded 18 sends for 9 jobs. The design tolerates duplicates; it does not pretend to remove them.
6. **Security gates must work in both time directions.** On 21 October a newly published advisory affected the *retained rollback* image, and the gate blocked the very release that remediated it. AD-28 keeps both scans and handles that case explicitly instead of weakening the gate (section 5.3).
7. **Keep misses visible.** Two-worker scaling (1.449× against 1.6×), Processing release time (10.1–10.3 min against 10 min) and queue wait (37 s against 20 s) are reported as measured. The targets were not changed after the fact.

6.3 Responsibilities for Use Cases and Design Problems

Each member led one specialism and reviewed the other two. For the presentation and assessment, each use case and design problem has a responsible member who explains it and answers questions on it. The allocation follows the team-agreed specialisms (effort ledger, PR #279).

Member	Specialism	Use cases	Design problem (section 7.3)
Guruprasath Gopal	Cloud and delivery: AWS, CI/CD	UC-6, UC-7	DP-1 Bounded cloud capacity and safe per-owner release
Ibrahim Mammadov	Backend services: Processing and Media	UC-1, UC-2, UC-4	DP-2 Durable asynchronous Processing handoff
Nguyễn Kim Long	Domain, contracts and verification	UC-3, UC-5	DP-3 Trusted owner identity and reusable public contract

6.4 Evidence and How to Reproduce It

Evidence	Location
Architecture: ADR register, DDD model, views, use-case catalogue	`docs/architecture/decisions.md`, `ddd.md`, `views.md`, `use-case-design.md`
Results against targets (table, CSV, figure)	`docs/results/actual-vs-target.md`; regenerate with `python scripts/dev/results_summary.py`
Scenario-by-scenario evidence index	`docs/evidence-index.md`
Cloud receipts (sanitized)	`docs/execution/aws/v1/evidence/`, `docs/results/engineering-20261020/`, `docs/results/engineering-20261021/`
Release identity and freeze	`docs/release/demo-release-manifest.md`, `docs/release/engineering-handoff.md`
Runbooks	`docs/execution/aws/v1/deploy-runbook.md`, `dlq-runbook.md`
Study guide (public)	https://collaboration95.github.io/dc2/
Run locally	`make setup-v1`, `make v1-up V1_RUN_ID=demo1`, `make v1-demo V1_RUN_ID=demo1 OPEN=1`
Tests	`make lint`, `make format-check`, `make test`; frontend `cd frontend && npm ci && npm test`
Live environment (until teardown after 16 Nov 2026)	Frontend https://d1e1dpefd6hp9x.cloudfront.net; API `https://nqxf2hclhi.execute-api.ap-southeast-1.amazonaws.com`
This report's sources	`project-documentation/final-report/` (Markdown sources, figures, build script)

6.5 AI Tool Declaration

The team used AI coding and writing assistants (Anthropic Claude models through Claude Code, and OpenAI GPT models for independent review) during the project:

- generating and refactoring code and tests under the team's direction;
- reviewing pull requests;
- drafting and proof-reading documentation, including this report;
- producing diagrams;
- running and recording verification steps.

Every result in this report comes from a retained test, CI run or cloud receipt, not from an AI statement. The team is responsible for the content and the quality of the submitted work.

7. Appendices

7.1 Use-Case Specifications and Traceability

Each specification is condensed from the use-case catalogue (`docs/architecture/use-case-design.md`), which links every test named here to its exact assertion. The personas are fictional planning personas (section 1.3).

UC-1 Register, log in and upload an owned video (Maya or Arun; web app or Publisher CLI)

- *Pre:* the account exists or is created; the caller owns a supported video of at most 100 MiB.
- *Normal:* authenticate; Media reserves an owner-scoped S3 grant; the client uploads directly to versioned S3; finalize verifies the bytes and pins the VersionId, then atomically adopts the Asset and stages `submit`; the API returns `202` with the asset and job ID.
- *Exceptions:* refused when the caller is anonymous, the type is unsupported, the declared size is too large, the owner is wrong, the request changed or the reservation expired. Missing, truncated, changed or unversioned bytes are not adopted and enqueue no work; a missing object returns a recoverable `409`.
- *Post:* one owned Asset, an immutable verified source and one durable command. A replay returns the same Asset without a second command.

UC-2 Process the video into playable outputs (platform worker, asynchronous)

- *Pre:* UC-1 committed a valid source and command.
- *Normal:* consume the command; create Job and Attempt; launch an encoder within the cap; claim under a DB-clock lease and task binding; verify the pinned source; encode the MP4 and the optional HLS ladder; commit the terminal state and result before acknowledging the work; update Media's projection.
- *Exceptions:* duplicates and reordering are fenced. A dependency, FFmpeg or deadline failure becomes a visible `failed`. An expired lease does not re-queue automatically. An HLS failure keeps the MP4. Poison messages follow the DLQ policy.
- *Post:* on success, exactly one primary output and optional HLS, scoped to the job and attempt. A failure is visible.

UC-3 Track status and play the result (owner; browser or CLI)

- *Pre:* the caller is authenticated and owns the Asset.
- *Normal:* Media reads the owner-filtered Asset and its local projection, and returns the status. When the job is done, it also returns a short-lived signed MP4 capability. The HLS playlist is served through Media, and segments redirect to signed S3 URLs.
- *Exceptions:* foreign and unknown IDs return not found. Stale results cannot replace a newer projection. An expired URL needs a fresh authorized read.
- *Post:* owner-only status and expiring playback access; no domain mutation.

UC-4 Retry from the stored source (owner; web or CLI)

- *Pre:* the caller owns a failed job whose accepted source remains available.
- *Normal:* a bodyless retry with an idempotency key. Media checks the owner, the failed projection and the pinned source. Processing serializes on the SubmissionGuard and creates a new Attempt. A replay returns that attempt. No bytes are uploaded again.
- *Exceptions:* a foreign or deleted job returns not found. A non-failed state or a changed request conflicts. A stale old-attempt worker or result cannot overwrite the current attempt.
- *Post:* the source identity is unchanged and one new Attempt is queued.

UC-5 Bulk-publish a folder and report results (Maya through Publisher; Jules as integrator)

- *Pre:* credentials and a folder of owner-held videos.
- *Normal:* authenticate; submit each eligible file through the published API; continue through the batch; report one outcome and job reference per file. `status`, `output` and `retry` share the public contract.
- *Exceptions:* one refused file does not hide later successes. Transport uncertainty preserves accepted IDs and request keys. Sensitive server error text is withheld.
- *Post:* per-file outcomes survive. Any failure makes the batch exit non-zero without losing successful references.

UC-6 Scale processing capacity independently (Sam, operator)

- *Pre:* a durable Processing backlog; Media is deployed independently.
- *Normal:* in V2-live, work sends wake the launcher, with a one-minute schedule as fallback. The launcher checks SQL and visible SQS demand, reserves slots and starts finite Fargate tasks, up to three. In V1, the operator started batches of 1 to 3.
- *Exceptions:* no visible work means no launch. An ambiguous RunTask reply keeps its reservation until resolved. An uncertain inventory fails closed. Excess work stays queued.
- *Post:* Processing capacity varied without any change to Media; concurrent triggers stayed within three slots.

UC-7 Release Processing on its own and demonstrate rollback (Sam through CD)

- *Pre:* the candidate has green required CI, an immutable and scanned image, migration evidence and a retained previous image.
- *Normal:* green Main CI updates the stack, and only changed owners release. The release validates the candidate, moves that owner's aliases and task definition, runs the smoke test and adopts the release. Media is unchanged.
- *Exceptions:* a failed gate stops promotion. A failed smoke restores the previous image. A failed restore leaves a durable recovery intent. Migrations are never reversed automatically.
- *Post:* success records the owner image and the smoke result; failure records the restore.

Traceability matrix. Analysis objects come from the shared analysis class model (Figure 16). Design classes come from the implemented design class model (Figure 15).

UC	Analysis objects	Design classes and modules	Sequence views	Key tests	Responsible
UC-1	Owner, UploadRequest, OwnedAsset	`User`, `UploadReservation`, `Asset`, `SourceVersion`, `Envelope`; `media/services/direct_intake.py`, `storage.py`, `media/messaging.py`	Figs 5, 17	`test_publisher_direct_post_encode_projection_and_ownership`, `test_finalize_pins_the_exact_version_it_verified`, `test_wrong_owner_cannot_finalize_or_observe_a_reservation`, `test_missing_object_is_retryable_not_adopted`	Ibrahim
UC-2	ProcessingJob, ProcessingAttempt, PlayableOutput	`Job`, `Attempt`, `SubmissionGuard`, `AttemptTask`, `Output`, `Source`; `processing/messaging.py`, `lifecycle.py`, `worker/tasks.py`, `media_engine/`	Figs 6, 17	`test_success_output_namespace_validation_and_atomic_terminal_result`, `test_a_failing_ladder_leaves_the_mp4_and_reports_no_ladder`	Ibrahim
UC-3	Owner, OwnedAsset, PlayableOutput	`ProcessingProjection`, `Actor`; `media/projections.py`, `media/api/v1_hls.py`, `storage.sign_output`	Figs 7, 17	`test_local_status_hls_and_signed_range_capability`, `test_wrong_owner_source_job_and_output_rejected`, `test_exported_handler_reads_owned_projection_without_peer`	Long
UC-4	ProcessingJob, ProcessingAttempt	`RetryRequest`, `Attempt`, `SubmissionGuard`, `RequestKey`; `media/services/v1_lifecycle.py:retry`, `processing/messaging.py:_retry_locked`	Fig 18; section 3.2.4 case 2	`test_publisher_retry_reuses_the_stored_source_after_a_transient_encode_failure`, `test_retry_idempotency_and_cancel_fence_old_and_current_attempts`, `test_retry_and_cancel_are_commands`	Ibrahim
UC-5	IntegrationBatch, UploadRequest	Publisher `cli.py`, `client.py`, `state.py`; OpenAPI documents in `docs/api/`	Fig 18	`test_cli_mixed_batch_keeps_every_file_outcome`, `test_module_entry_and_import_boundary`, `test_public_schema_conformance_and_explicit_ca`	Long
UC-6	Operator, ProcessingCapacity	`LauncherControl`, `LaunchReservation`, `AttemptTask`; `processing/launcher.py`, `launcher_trigger.py`	Figs 6, 19	`test_always_on_concurrent_wakes_never_exceed_the_cap_without_a_session_end`, `test_a_backlog_of_ten_launches_at_most_three_and_later_ticks_add_none`, `test_a_lost_runtask_response_is_resolved_by_token_without_a_second_launch`	Guruprasath
UC-7	Operator, ReleaseCandidate	`Planner`, `Journal`, `CLITransport`; `scripts/aws/v1/release.py`, `deploy.py`, `scripts/ci/cd.sh`	Fig 19; section 3.2.4 case 3	`test_owner_release_applies_a_real_migration_and_failed_candidate_restores_aliases`, `test_pending_intent_before_mutation_and_real_bad_candidate_restore`, `test_owners_are_released_only_when_their_images_build_inputs_changed`	Guruprasath

7.2 Analysis and Design Models

Transition from analysis to design. Each analysis responsibility became a design structure (static change) and an interaction protocol (dynamic change). The table records the affected use cases and the rationale.

Analysis responsibility	Affected UCs	Static design (classes, data)	Dynamic design (interactions)	Rationale
Owner submits an asset and follows its job	UC-1, UC-3	Media owns `User`, `Asset`, `UploadReservation`, `SourceVersion` and a local `ProcessingProjection`; Processing owns `Job` and `Attempt`; UUIDs, not foreign keys, cross the boundary	Reserve → direct upload → finalize; result snapshots update the projection	Two logical databases with their own roles; no runtime dependency between the services
Owner identity and authorization	UC-1, UC-3, UC-4, UC-5	`Actor` value type; Media authorizer; Gateway adapter	The authorizer reads the current user row per request; the adapter strips caller headers	No shared JWT secret and no caller-controlled owner header at Processing
Upload bytes and source identity	UC-1, UC-5	`UploadReservation` with a pin check constraint; `Source` descriptor	Presigned POST to S3; verify outside SQL locks; adopt in one transaction	Bytes never pass through Lambda; the verified source cannot be replaced
Cross-context asynchronous work	UC-1, UC-2, UC-4	`message_outbox`, `message_inbox` in each database; `Envelope` v1	Stage in the domain transaction; publish after commit; consumer commits inbox and state before ACK	SQL and SQS are not one atomic resource; at-least-once delivery is absorbed
Retry, worker failure and output	UC-2, UC-4	`RetryRequest`, `AttemptTask`, lease columns, `Output`	A new attempt per retry; DB-clock leases fence old workers	No duplicate terminal writes; no hidden automatic requeue
Operator capacity and release	UC-6, UC-7	`LauncherControl`, `LaunchReservation`; release journal and pending intent	SQL-reserved RunTask with a stable token; per-owner alias move, smoke and restore	Capacity and code change independently; bounded cost

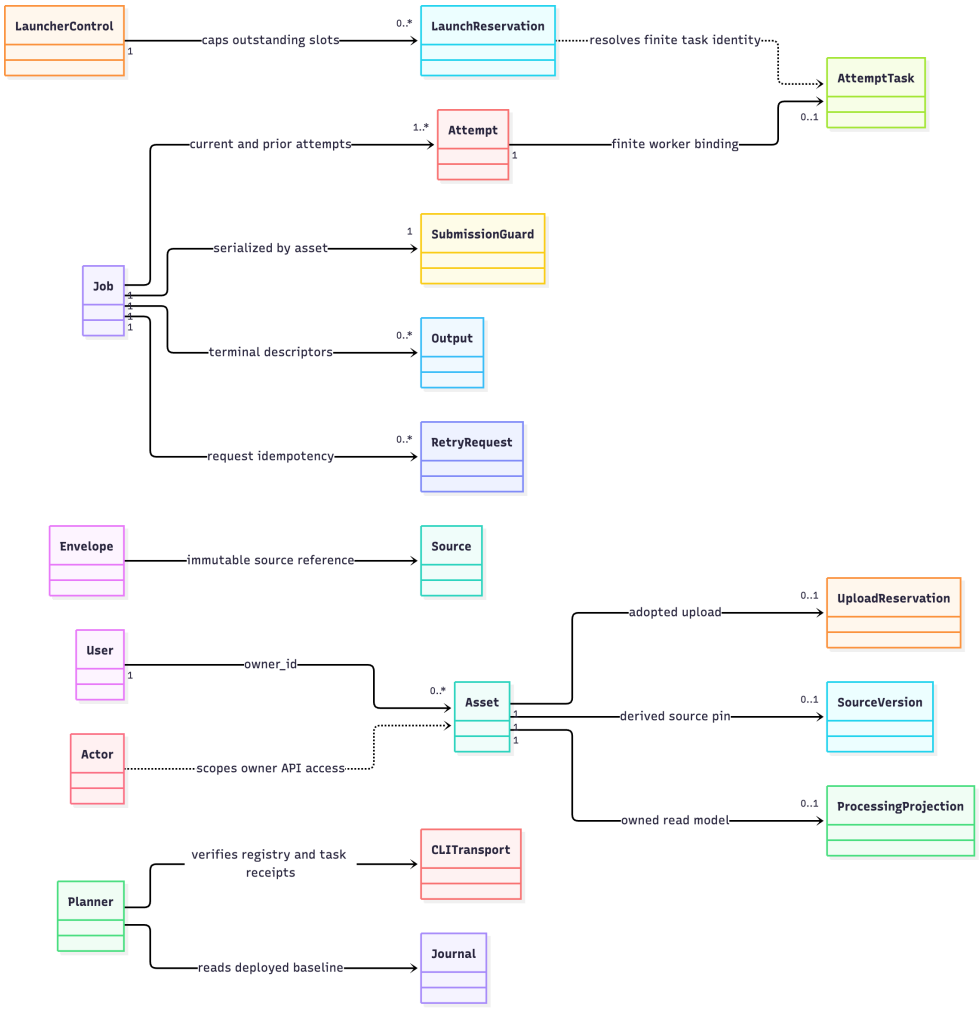


Figure 15. Common implemented design class model for UC-1 to UC-7 (domain, contract and operator-tool types).

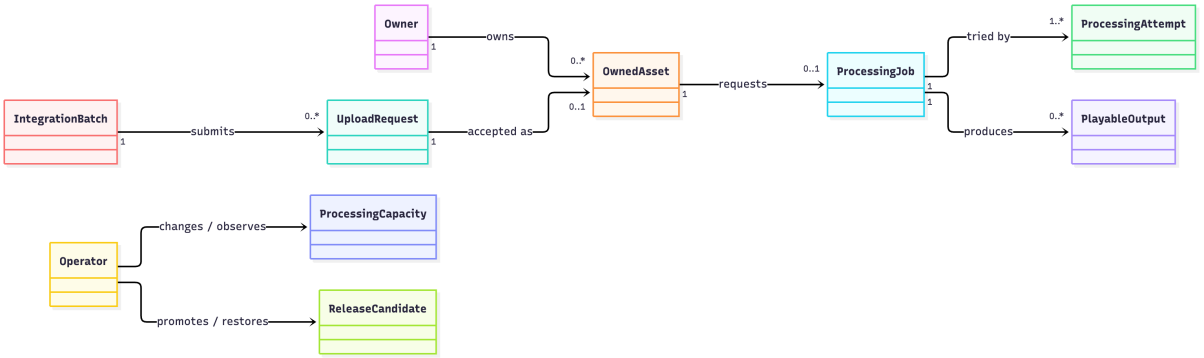


Figure 16. Shared analysis class model (conceptual objects only).

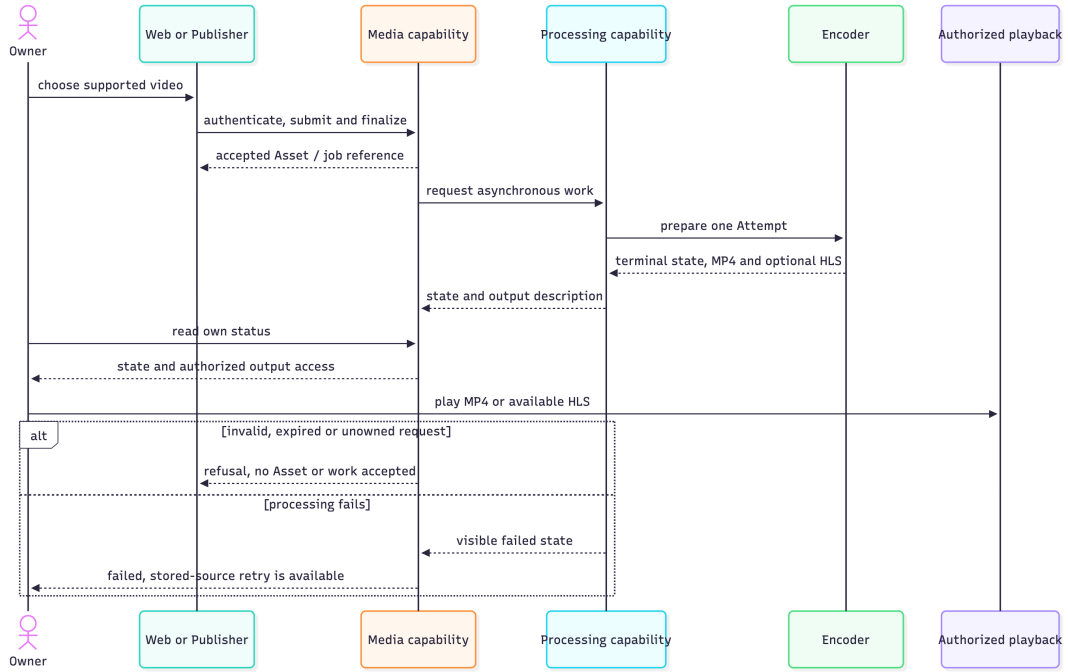


Figure 17. Analysis sequence, UC-1 to UC-3: the core workflow.

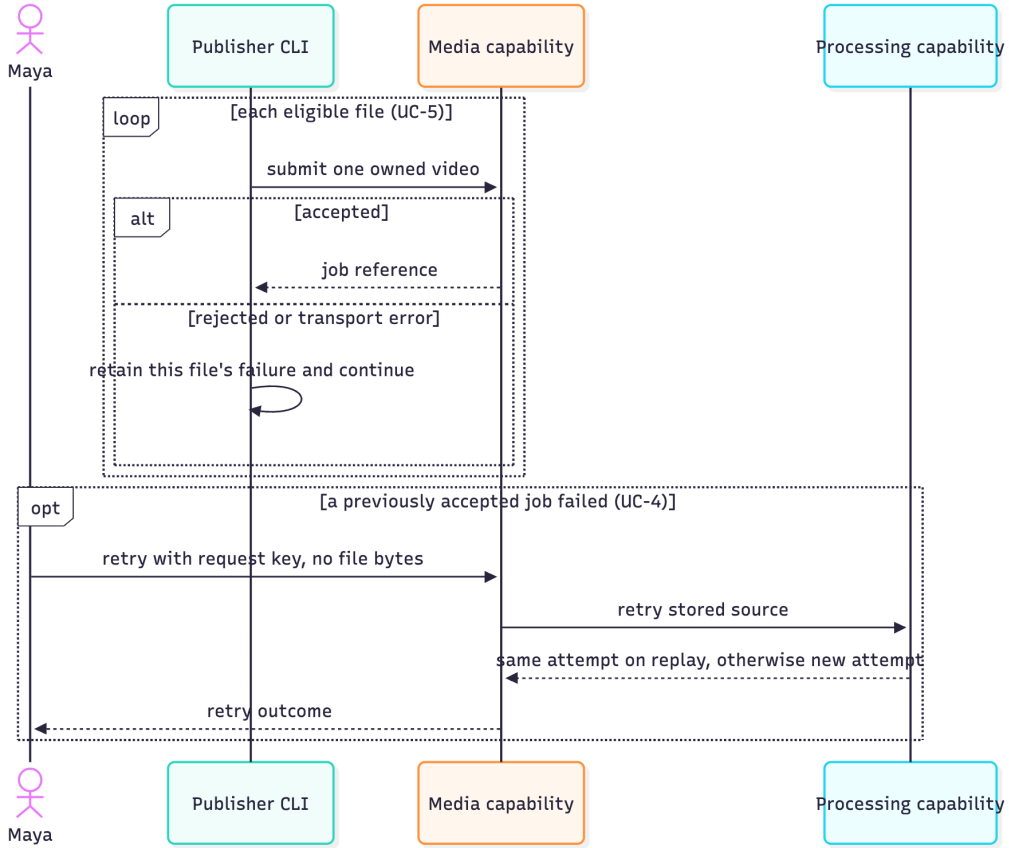


Figure 18. Analysis sequence, UC-4 and UC-5: stored-source retry and Publisher integration.

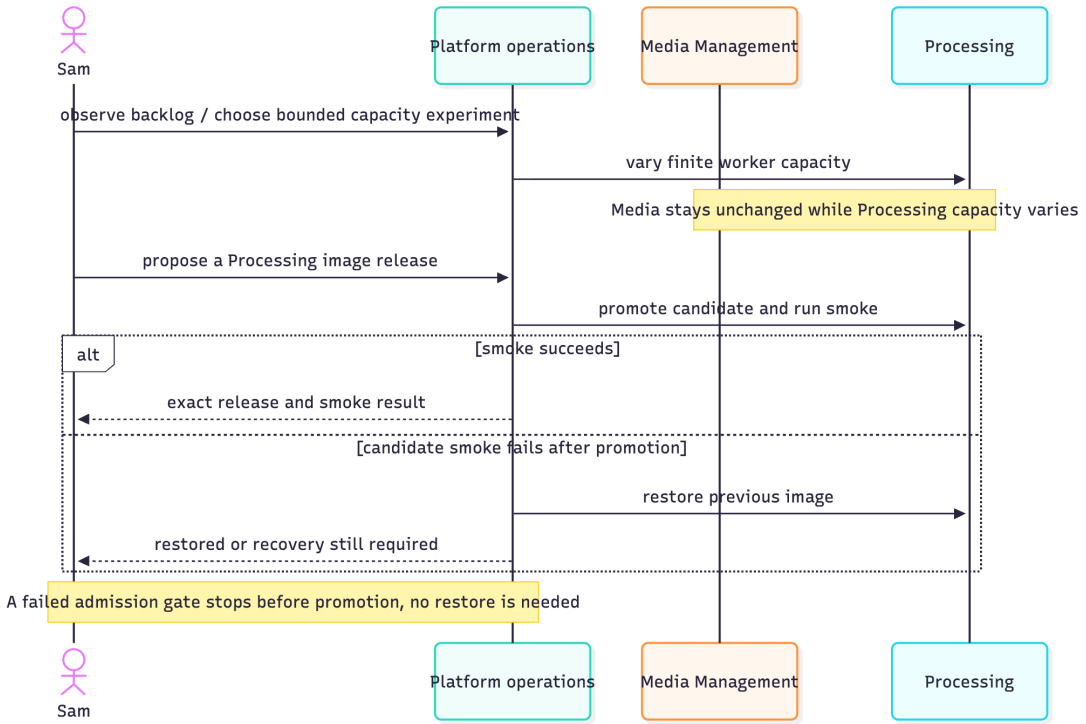


Figure 19. Analysis sequence, UC-6 and UC-7: capacity and release.

The implemented (design-level) sequences for the same three groups name every Lambda, queue and table. They are too wide to print legibly. They are kept as source in the catalogue and rendered in `docs/architecture/use-case-design.md`. The compact design sequences in Figures 5–7 cover the core workflow.

7.3 Design Problems

The three significant design problems are listed below. Each one covers the problem, the alternatives, the decision, a before-and-after view, the resulting behaviour and the evidence. The “before” is the submitted proposal design or the V1 operating mode. These are as-built design decisions, not claims that a named GoF pattern was chosen. They do use well-known architectural patterns: transactional outbox and inbox, anti-corruption layer, and lease-based fencing.

7.3.1 DP-1 Bounded cloud capacity and safe per-owner release (Guruprasath Gopal)

- **Problem.** The environment must be usable at any time while idle cost and the blast radius of a release stay small. Processing must change without releasing Media. An ambiguous cloud response must never create extra workers, or falsely report a successful rollback.
- **Alternatives.**
    - Keep per-run stacks and booked sessions: slow, at about 25 minutes to deploy and 25–40 minutes to tear down.
    - Keep one long-open session: a booking under another name.
    - Release all code with a full stack update: couples the domains.
    - Use mutable image tags: loses the tested identity.
    - Add a separate staging environment: out of scope for the time available.
- **Decision.** AD-27 with AD-16. One persistent stack is updated after green Main CI. Only owners whose image inputs changed are released, by immutable digest, with a smoke test and automatic restore. A SQL-backed launcher reserves at most three encoder slots, and a stable RunTask token survives lost replies.
- **Behaviour.** A failed preflight or smoke is never counted as a promotion. A journal lock and a `pending_release` intent precede any mutation. The previous image is retained for restore and, under AD-28, is scanned before promotion.
- **Evidence.** The tests named under UC-6 and UC-7 (section 7.1). **[AWS]**: 0 → 3 → 0 scaling, the 9-clip backlog, and the bad-candidate restore in run 39271754470.

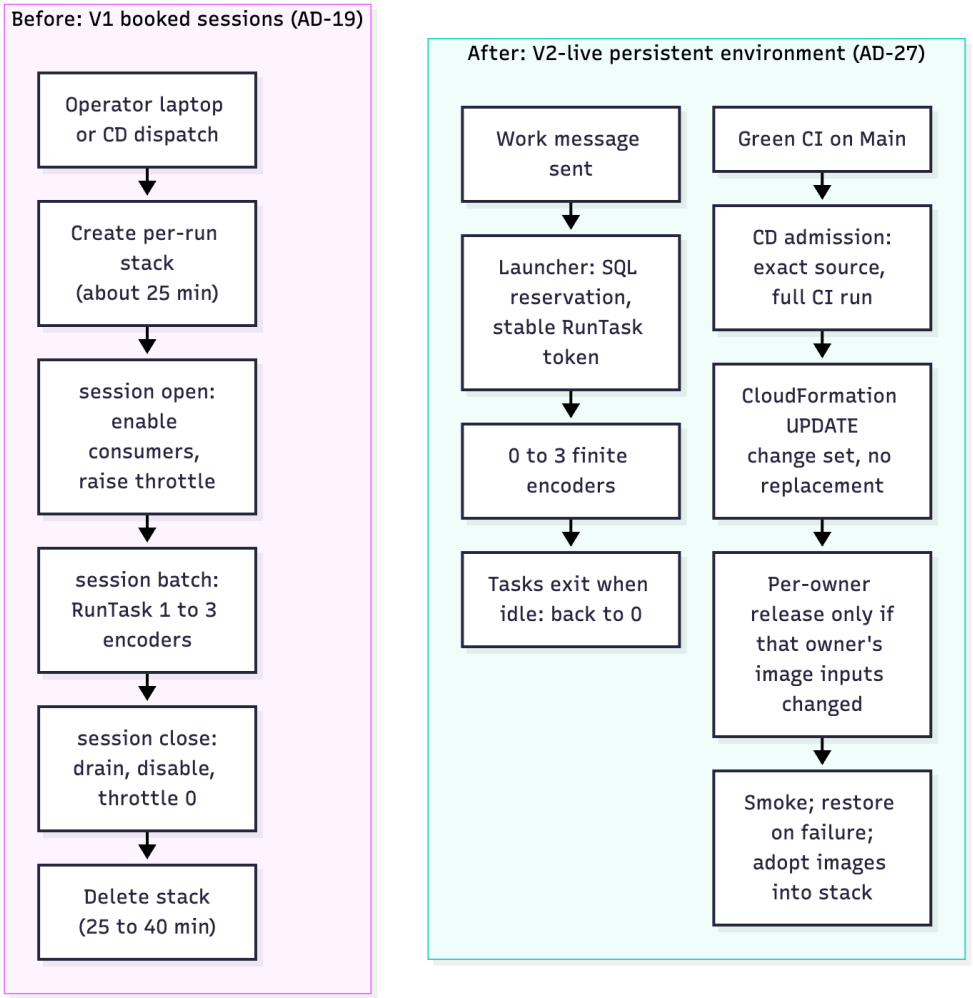


Figure 20. DP-1 before and after: booked per-run sessions (left) and the persistent, automatically updated environment with SQL-reserved finite encoders (right).

7.3.2 DP-2 Durable asynchronous Processing handoff (Ibrahim Mammadov)

- **Problem.** An upload must stay accepted if Processing or SQS is temporarily unavailable. No Job may exist without a durable command, and no work may be published before its state commits.
- **Alternatives.**
  - Keep peer HTTP with Redis/RQ: couples availability, and needs an always-on broker.
  - Perform an unprotected SQL-plus-SQS dual write from the API: can lose or orphan messages.
  - Add another broker or a workflow service: more operations.
- **Decision.** AD-11 with AD-21: a transactional outbox and inbox in each database, SQS between them, and a version-fenced projection in Media.
- **Behaviour.**
  - The Media API role cannot send to SQS; a repair publisher claims rows and releases SQL locks before network I/O.
  - Duplicates are possible. Consumers commit inbox, state and follow-on outbox rows before ACK.
  - Reordering is handled by `state_version`, attempt, guard and lease fences.

    - `repair_work` rebuilds missing queue entries from SQL.
- **Evidence.** `test_messaging_sql.py`: lost-send replay, an SQS-emulator end-to-end run, and stale-attempt fencing **[Local]**. **[AWS]**: the Processing-down isolation run, and the DLQ poison message.

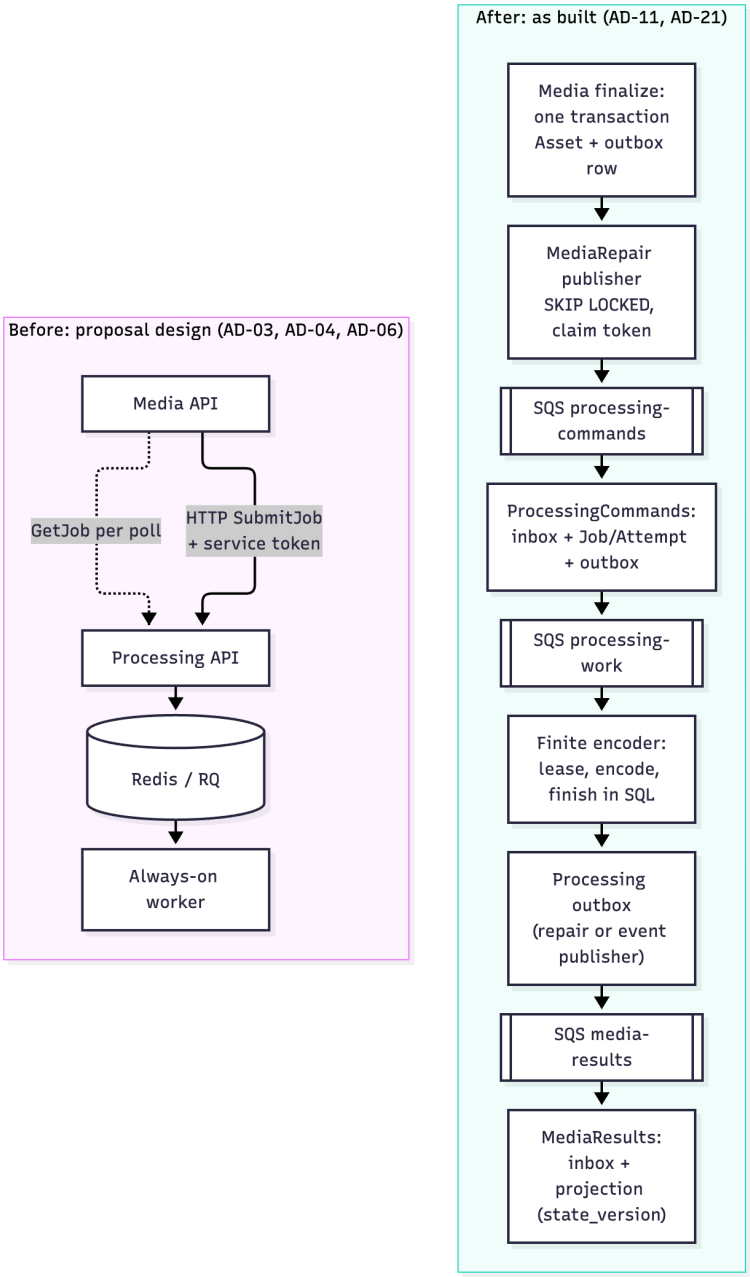


Figure 21. DP-2 before and after: synchronous SubmitJob with Redis/RQ (left) and the outbox → SQS → inbox chain with Media's projection (right).

7.3.3 DP-3 Trusted owner identity and reusable public contract (Nguyễn Kim Long)

- **Problem.** An independent client (Jules) and Processing need the same owner decision. Neither may trust a caller-supplied owner header, and Processing must not get Media's JWT secret or user table.
- **Alternatives.**
    - Trust `X-Owner-Id`/`X-Role` headers.
    - Give the JWT secret and user lookup to both services.

    - Create a separate identity provider or service.
- **Decision.** AD-09 with AD-20 and AD-24. A Media-owned authorizer passes a validated `Actor`. The Gateway adapter strips the public identity headers, and Processing filters by the trusted owner. IAM and queue policies, not the envelope schema, establish producer and consumer authority. The OpenAPI and `contracts/` documents are versioned, so a client like Publisher needs no service imports or credentials.
- **Behaviour.** Forged headers change nothing; a foreign job returns 404; direct queue sends are refused.
- **Evidence.** `test_local_gateway.py` (forged-header and anonymous-denial tests), `test_contract_fixtures.py` (trusted owner propagation) and the Publisher import-boundary test **[Local]**. **[AWS]**: `forged-identity`, `anonymous-denied`, `sqs-direct-send`, and the #229 cross-owner negatives.

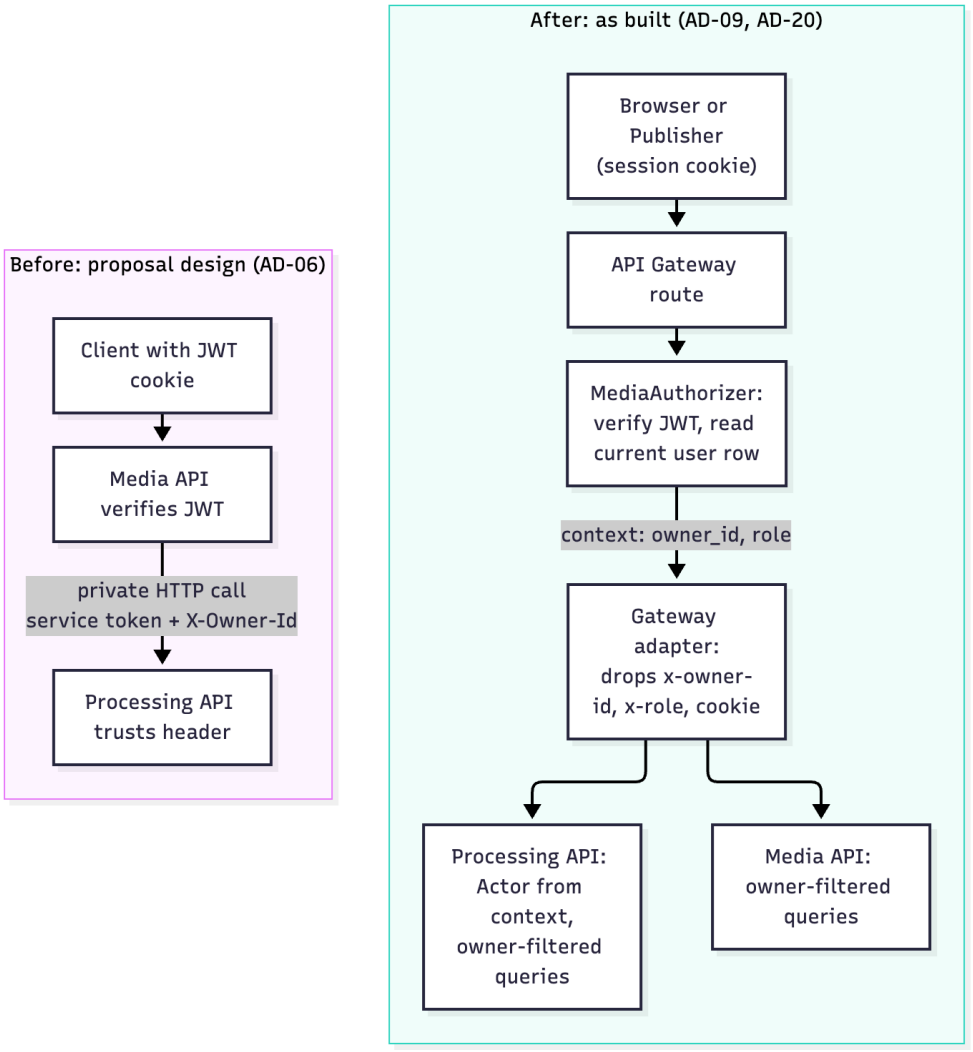


Figure 22. DP-3 before and after: service token with a forwarded owner header (left) and the Gateway-trusted actor from the Media authorizer (right).

7.4 Glossary

Term	Meaning
ACU	Aurora capacity unit; Serverless v2 scales between 0 and 2 ACU here
ADR / AD-nn	Architecture decision record and its identifier in `docs/architecture/decisions.md`
AISS-nn	Outstanding architectural issue (section 3.4)
Attempt	One numbered try at a job; never reused
CD / CI	Continuous delivery / continuous integration (GitHub Actions)
`dc2-live`	The persistent V2-live AWS environment (stack name)
DLQ	Dead-letter queue: receives a message after five failed receives
Inbox / outbox	Per-database tables that deduplicate received messages / stage outgoing messages in the same transaction as the state change
Lease	A worker's time-limited right to finish an attempt, judged on the database clock
OIDC	OpenID Connect: lets GitHub Actions assume an AWS role without stored keys
Projection	Media's own read model of Processing state, fenced by `state_version`
Release (per owner)	Moving one service's Lambda aliases and task definition to a new image digest, with smoke and restore
Seed / producer / consumer	The platform's core asset (an owned video and its job) / who submits it / who uses the result
V1 / V2-live	Per-run booked-session stacks (to 18 Oct) / one persistent environment (from 18 Oct, AD-27)
VersionId	The S3 object version that pins an accepted source